1 Streszczenie

1.1 Streszczenie

Praca inżynierska dotyczy projektu i implementacji inteligentnego systemu zarządzania zasobami cyfrowymi z wykorzystaniem wyszukiwania semantycznego. System ten ma na celu umożliwienie efektywnego przechowywania, organizowania oraz przeszukiwania plików cyfrowych w różnych formatach. Kluczowym elementem rozwiązania jest zastosowanie mechanizmów inteligentnego wyszukiwania, które pozwalają użytkownikowi odnajdywać dokumenty nie tylko na podstawie słów kluczowych, ale również znaczenia zapytań oraz podobieństwa treści.

System obejmuje funkcje dodawania i zarządzania zasobami, ich klasyfikacji za pomocą kategorii i tagów oraz edycji metadanych. Istotną rolę odgrywa również kontrola wersji, wykrywanie duplikatów oraz możliwość udostępniania plików innym użytkownikom. Rozwiązanie wspiera także filtrowanie i sortowanie danych, co ułatwia ich organizację i szybkie odnajdywanie potrzebnych informacji.

Dodatkowo system umożliwia pobieranie zasobów z opcjonalną konwersją formatu oraz automatyczne generowanie miniaturek dokumentów. Całość uzupełniają mechanizmy uwierzytelniania i autoryzacji użytkowników, które zapewniają bezpieczny dostęp do zasobów. Projekt uwzględnia zarówno aspekty funkcjonalne, jak i wymagania jakościowe, takie jak wydajność, bezpieczeństwo i użyteczność systemu.

Słowa kluczowe: system zarządzania dokumentami, DMS, wyszukiwanie semantyczne, wektory osadzeń, baza wektorowa, pgvector, PostgreSQL

1.2 Abstract

The engineering thesis focuses on the design and implementation of an intelligent digital asset management system utilizing semantic search. The system is intended to enable the efficient storage, organization, and searching of digital files in various formats. A key element of the solution is the use of intelligent search mechanisms that allow users to find documents not only based on keywords but also on the meaning of queries and content similarity. The system includes functions for adding and managing resources, classifying them using categories and tags, and editing metadata. Version control, duplicate detection, and the ability to share files with other users also play a significant role. The solution also supports filtering and sorting data, which facilitates organization and quick retrieval of needed information.

Additionally, the system allows for downloading resources with optional format conversion and automatic generation of document thumbnails. The system is complemented by user authentication and authorization mechanisms, which ensure secure access to resources. The project takes into account both functional aspects and quality requirements, such as performance, security, and usability of the system.

Keywords: document management system, DMS, semantic search, embeddings, vector database, pgvector, PostgreSQL

Spis treści

1 Streszczenie **1**

1.1 Streszczenie 1

1.2 Abstract 1

2 Wstęp **4**

3 Cel, zakres i podział pracy **5**

3.1 Cel pracy 5

3.2 Zakres pracy 5

3.3 Podział pracy 6

3.3.1 Mateusz 6

3.3.2 Mateusz 6

3.3.3 Mateusz 6

4 Analiza rynku **7**

4.1 Paperless-ngx 7

4.1.1 Główne cechy 7

4.1.2 Ograniczenia 7

4.2 OpenKM 9

4.2.1 Główne cechy 9

4.2.2 Ograniczenia 10

4.3 M-Files 11

4.3.1 Główne cechy 11

4.3.2 Ograniczenia 11

4.4 Wnioski 12

5 Projekt systemu **14**

5.1 Specyfikacja wymagań funkcjonalnych 14

5.1.1 Dodawanie zasobów 14

5.1.2 Wyszukiwanie zasobów 14

5.1.3 Filtrowanie i sortowanie zasobów 14

5.1.4 Zarządzanie kategoriami i tagami 15

5.1.5 Zarządzanie zasobami 15

5.1.6 Udostępnianie i zarządzanie udostępnieniami zasobów 15

5.1.7 Kontrola wersji i wykrywanie duplikatów zasobów 15

5.1.8 Pobieranie zasobów z opcjonalną konwersją formatu 15

5.1.9 Generowanie miniaturek zasobów . . . 15
5.1.10 Autoryzacja i uwierzytelnianie użytkowników . . . 15
5.2 Specyfikacja wymagań niefunkcjonalnych . . . 16
5.2.1 Wydajność (dla bazy zawierającej do 1000 dokumentów) . . . 16
5.2.2 Pojemność . . . 16
5.2.3 Bezpieczeństwo . . . 16
5.2.4 Kompatybilność . . . 16
5.2.5 Użyteczność . . . 16
5.3 Hierarchiczny model funkcji . . . 16
5.4 Historyjki użytkowników . . . 18
5.4.1 Rejestracja i uwierzytelnianie użytkowników . . . 18
5.4.2 Dodawanie dokumentów . . . 18
5.4.3 Przeglądanie dokumentów . . . 18
5.4.4 Wyszukiwanie dokumentów . . . 18
5.4.5 Zarządzanie dokumentami i metadanymi . . . 19
5.4.6 Pobieranie i konwersja dokumentów . . . 19
5.4.7 Udostępnianie dokumentów . . . 19
5.5 Scenariusze przypadków użycia . . . 19
5.5.1 Pobieranie dokumentu z konwersją do formatu PDF. . . . 19
5.5.2 Udostępnianie dokumentu po wyszukaniu semantycznym . . . 20
5.6 Diagramy przypadków użycia . . . 21
5.7 Diagramy procesów biznesowych . . . 25
5.8 Diagramy przepływu danych . . . 33
5.9 Diagram stanów . . . 35
5.10 Diagram bazy danych . . . 37
5.11 Projekt interfejsu graficznego . . . 39

Bibliografia **40**

2 Wstęp

W dobie powszechnej cyfryzacji oraz dynamicznego rozwoju technologii informatycznych, efektywne zarządzanie informacją stało się jednym z kluczowych wyzwań stojących zarówno przed organizacjami, jak i użytkownikami indywidualnymi. Gwałtowny przyrost wolumenu generowanych danych sprawia, że tradycyjne metody przechowywania dokumentów w strukturach katalogów systemowych stają się niewystarczające. Prowadzi to do powstawania barier informacyjnych, trudności w utrzymaniu spójności wersji dokumentów oraz znacznego wydłużenia czasu potrzebnego na dotarcie do poszukiwanej wiedzy. Rozwiązaniem tych problemów są systemy zarządzania dokumentami klasy DMS (ang. *Document Management System*), które automatyzują procesy archiwizacji, kategoryzacji i kontroli dostępu do plików.

Tradycyjne systemy DMS opierają swoje mechanizmy wyszukiwania na sztywnych strukturach metadanych oraz klasycznym indeksowaniu pełnotekstowym. Podejście to, choć skuteczne w przypadku znajomości dokładnych fraz lub nazw plików, okazuje się bezużyteczne, gdy użytkownik formułuje zapytania w języku naturalnym lub poszukuje dokumentów powiązanych kontekstowo bądź znaczeniowo. Przeprowadzona analiza rynku wykazała istotną lukę technologiczną. Istniejące rozwiązania open-source, takie jak Paperless-ngx, oferują lekkość wdrożenia, lecz są pozbawione zaawansowanej analizy semantycznej treści. Z kolei rozbudowane systemy korporacyjne, jak M-Files, wprowadzają zaawansowane mechanizmy sztucznej inteligencji, jednak wiążą się z wysokim progiem wejścia, skomplikowaną konfiguracją oraz silną zależnością od zewnętrznych, zamkniętych ekosystemów chmurowych.

Niniejsza praca inżynierska stanowi odpowiedź na zidentyfikowane ograniczenia rynkowe poprzez zaprojektowanie, implementację oraz przetestowanie inteligentnego systemu DMS wykorzystującego nowoczesne metody wyszukiwania semantycznego. Istotą rozwijanego rozwiązania jest integracja tradycyjnych funkcji platformy DMS, takich jak kontrola wersji dokumentów, wykrywanie duplikatów na podstawie zawartości oraz taksonomia za pomocą kategorii i tagów.

Kluczowym elementem innowacji projektowanego systemu DMS jest autorska architektura bazy danych, która w ramach jednej instancji silnika PostgreSQL łączy cechy magazynu relacyjnego i wektorowego. Wykorzystanie rozszerzenia *pgvector* pozwala na składowanie matematycznych reprezentacji tekstu w postaci embeddingów o długości 2048 komponentów (`halfvec(2048)`), na które nakładane są wydajne indeksy HNSW. Dzięki temu system umożliwia błyskawiczne realizowanie zapytań opartych na podobieństwie cosinusowym, pozwalając na odnajdywanie dokumentów na podstawie ich rzeczywistego znaczenia oraz wyszukiwanie plików podobnych do wskazanego zasobu źródłowego.

Mając na uwadze rygorystyczne wymagania niefunkcjonalne dotyczące wydajności i pojemności, wszelkie operacje kosztowne procesowo zostały odizolowane od głównego wątku aplikacji i przeniesione do warstwy asynchronicznej. Procesy takie jak ekstrakcja tekstu, generowanie wektorów osadzenia przy użyciu zewnętrznego modułu AI Inference, konwersja formatów (np. transformacja plików DOCX do formatu PDF) oraz renderowanie graficznych miniaturek dokumentów są zarządzane przez dedykowaną kolejkę zadań systemowych (`document_job_queue`). Bezpieczeństwo i skalowalność systemu zapewnia fizyczne odseparowanie metadanych od samych dokumentów, które wraz

3 Cel, zakres i podział pracy

3.1 Cel pracy

Celem pracy jest zaprojektowanie, implementacja oraz przetestowanie systemu zarządzania dokumentami (Document Management System), umożliwiającego przechowywanie, organizację, wyszukiwanie oraz udostępnianie zasobów cyfrowych. Projektowany system ma wspierać użytkowników w zarządzaniu dokumentami poprzez wykorzystanie zarówno klasycznych mechanizmów wyszukiwania, jak i wyszukiwania semantycznego opartego na reprezentacji wektorowej treści dokumentów.

System ma umożliwiać gromadzenie dokumentów w różnych formatach, zarządzanie ich metadanymi, kategoryzację, tagowanie oraz kontrolę wersji. Dodatkowo zasoby powinny być prezentowane w sposób ułatwiający ich identyfikację i przeglądanie, między innymi poprzez automatyczne generowanie miniaturek dokumentów. Użytkownik powinien mieć możliwość wyszukiwania dokumentów na podstawie zapytań w języku naturalnym oraz odnajdywania dokumentów podobnych pod względem treści i zawrtości wizualnej.

System ma również zapewniać możliwość udostępniania zasobów za pomocą dedykowanych linków dostępowych oraz zapewniać możliwość konwersji dokumentów pomiędzy wybranymi formatami, co zwiększy elastyczność wykorzystania zgromadzonych zasobów.

Rezultatem pracy będzie działający prototyp systemu składający się z warstwy serwerowej i aplikacji interetowej, umożliwiający realizację zdefiniowanych wymagań funkcjonalnych i niefunkcjonalnych.

3.2 Zakres pracy

Zakres pracy obejmuje:

- analizę rynku istniejących systemów zarządzania dokumentami pod kątem dostępnych funkcjonalności oraz wykorzystywanych technologii,
- określenie wymagań funkcjonalnych i niefunkcjonalnych projektowanego systemu,
- opracowanie historyjek użytkowników oraz scenariuszy przypadków użycia,
- wykonanie diagramów procesów biznesowych BPMN,
- opracowanie diagramów projektowych, w tym diagramu przepływu danych (DFD), diagramu związków encji (ERD) oraz diagramu stanów,
- zaprojektowanie interfejsu użytkownika dla aplikacji webowej,
- dobór technologii wykorzystywanych do implementacji systemu,
- dobór modelu sztucznej inteligencji do generowania reprezentacji wektorowej dokumentów,
- implementację warstwy serwerowej odpowiedzialnej za zarządzanie dokumentami, przetwarzanie plików oraz realizację wyszukiwania semantycznego,
- implementację aplikacji webowej,
- implementację mechanizmów przechowywania dokumentów, zarządzania metadanymi, wersjonowania oraz udostępniania zasobów,

- implementację mechanizmów wyszukiwania dokładnego i semantycznego z wykorzystaniem reprezentacji wektorowej dokumentów,
- przeprowadzenie testów funkcjonalnych systemu,
- przeprowadzenie testów wydajnościowych systemu,
- analizę uzyskanych wyników oraz sformułowanie wniosków końcowych.

3.3 Podział pracy

3.3.1 Mateusz

- Streszczenie (rozdział 1)
- Wstęp (rozdział 2)
- Cel, zakres i podział pracy (rozdział 3)
- Specyfikacja wymagań niefunkcjonalnych (podrozdział 5.2)
- Scenariusze przypadków użycia (podrozdział 5.5)
- Diagramy przypadków użycia (podrozdział 5.6)
- Diagramy przepływu danych (podrozdział 5.8)
- Projekt interfejsu graficznego (podrozdział 5.11)

3.3.2 Mateusz

- Hierarchiczny model funkcji (podrozdział 5.3)
- Diagramy przypadków użycia (podrozdział 5.6)

3.3.3 Mateusz

- Analiza rynku istniejących systemów zarządzania dokumentami (rozdział 4)
- Specyfikacja wymagań funkcjonalnych (podrozdział 5.1)
- Historyjki użytkowników (podrozdział 5.4)
- Diagramy procesów biznesowych (podrozdział 5.7)
- Diagram stanów (podrozdział 5.9)
- Diagram bazy danych (podrozdział 5.10)

4 Analiza rynku

Rosnąca ilość danych i dokumentów cyfrowych sprawia, że organizacje muszą zmieniać podejście do ich przechowywania, organizacji i przetwarzania. Tradycyjne metody zarządzania informacją przestają być wystarczające, co prowadzi do rosnącego zapotrzebowania na systemy klasy DMS (Document Management System). Rozwiązania te umożliwiają nie tylko archiwizację dokumentów, ale również ich wersjonowanie, wyszukiwanie, kontrolę dostępu oraz automatyzację obiegu informacji.

Rynek systemów DMS jest obecnie bardzo zróżnicowany i dynamicznie się rozwija, obejmując zarówno lekkie, często otwartoźródłowe aplikacje przeznaczone dla małych firm lub indywidualnych użytkowników, jak i zaawansowane platformy klasy enterprise, oferujące szeroki zakres funkcji integracyjnych, skalowalność oraz wysoki poziom bezpieczeństwa. W niniejszej analizie zestawiono główne podejścia do projektowania systemów DMS: rozwiązania minimalistyczne, skoncentrowane na prostocie i łatwości wdrożenia, oraz rozbudowane systemy korporacyjne, dostosowane do potrzeb dużych organizacji i złożonych procesów biznesowych.

4.1 Paperless-ngx

Paperless-ngx to nowoczesny, otwartoźródłowy system zarządzania dokumentami, ukierunkowany na automatyzację procesu digitalizacji, klasyfikacji i archiwizacji plików. Projekt jest aktywnie rozwijany przez społeczność, która dostarcza również dodatkowe aplikacje desktopowe i mobilne, rozszerzające jego możliwości. System został zaprojektowany z myślą o użytkownikach indywidualnych oraz małych organizacjach, oferując prostotę wdrożenia przy jednoczesnym zachowaniu istotnych funkcji zarządzania dokumentami.

Stos technologiczny: Python — Django — Angular — PostgreSQL

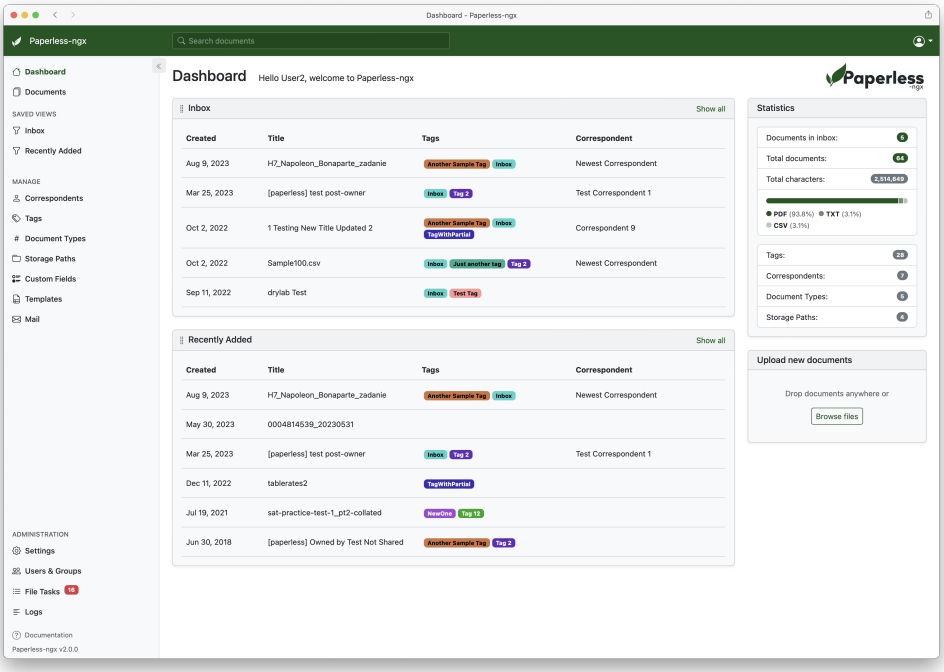
4.1.1 Główne cechy

- Automatyczne przetwarzanie dokumentów z wykorzystaniem technologii OCR, umożliwiające ekstrakcję treści z plików graficznych i PDF.
- Mechanizmy inteligentnego tagowania dokumentów oparte na modelach klasyfikacyjnych.
- Integracja z narzędziami do akwizycji dokumentów, takimi jak poczta elektroniczna oraz monitorowane katalogi systemowe.
- Wsparcie dla wyszukiwania pełnotekstowego w zgromadzonych zasobach.
- Przeglądarkowy interfejs użytkownika zaprojektowany z naciskiem na użyteczność i prostotę obsługi.
- Możliwość lokalnego wdrożenia systemu (self-hosting), zapewniająca pełną kontrolę nad danymi.
- Rozszerzalna architektura systemu dzięki udokumentowanemu interfejsowi API.

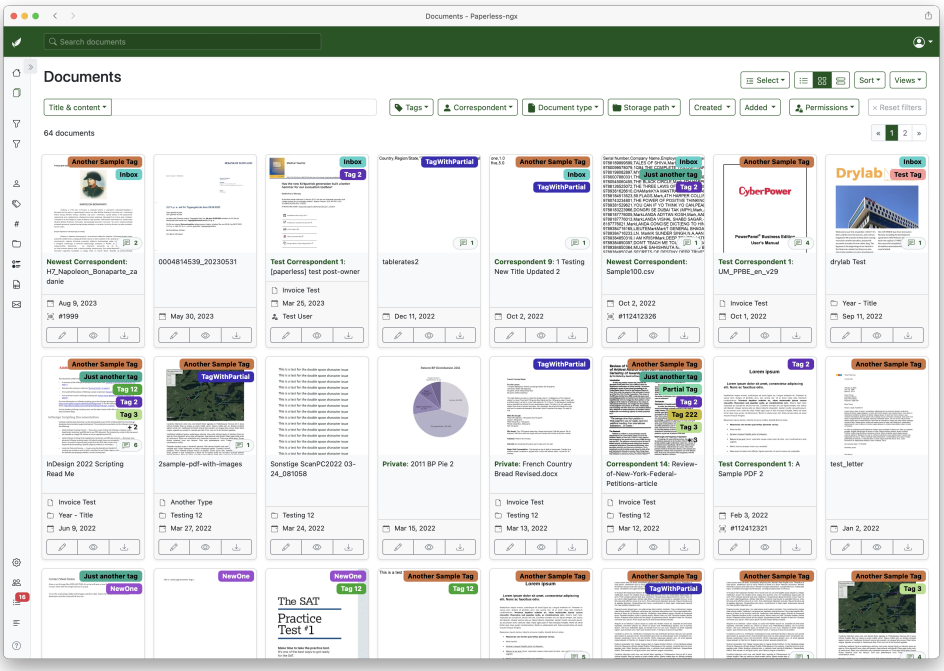
4.1.2 Ograniczenia

- Brak zaawansowanego mechanizmu zarządzania przepływem pracy (workflow), w szczególności ograniczone możliwości modelowania procesów biznesowych.

- Ograniczony system kontroli dostępu, niewspierający szczegółowych (granularnych) polityk uprawnień.
- Brak natywnej obsługi zaawansowanych integracji z zewnętrznymi systemami informatycznymi.
- Ograniczone możliwości definiowania i konfiguracji schematów metadanych dokumentów.
- Brak rozbudowanych mechanizmów wersjonowania dokumentów, istotnych w środowiskach wieloetapowej edycji.



Rys. 4.1. Paperless-ngx - Panel główny



Rys. 4.2. Paperless-ngx - Spis dokumentów

4.2 OpenKM

OpenKM to zaawansowany system zarządzania dokumentami klasy enterprise, dostępny zarówno w wersji open-source, jak i komercyjnej. Platforma jest przeznaczona dla średnich i dużych przedsiębiorstw, które wymagają rozbudowanych mechanizmów zarządzania treścią, obiegiem dokumentów oraz integracji z istniejącą infrastrukturą IT. System wspiera pełen cykl życia dokumentu, obejmujący jego rejestrację, przetwarzanie, dystrybucję i archiwizację, zgodnie z przyjętymi standardami, takimi jak ISO-15489.

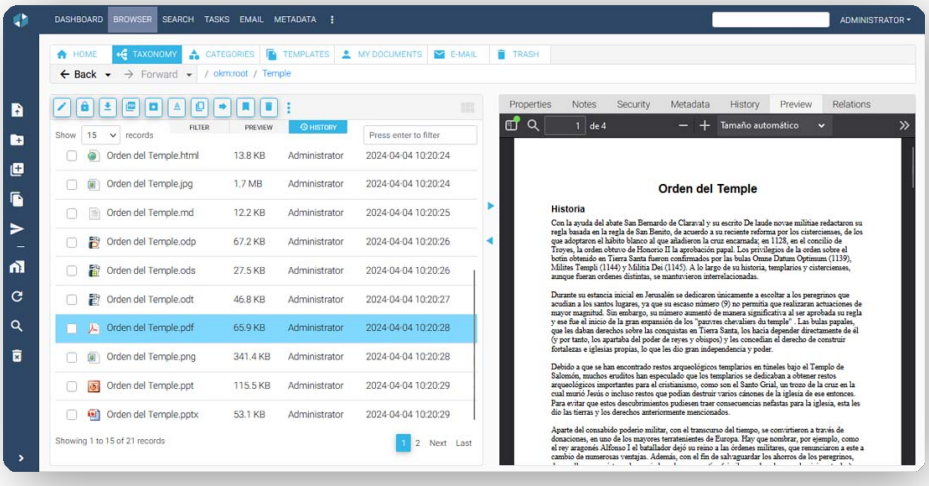
Stos technologiczny: Java — GWT — Hibernate — Apache Tomcat — MySQL / PostgreSQL

4.2.1 Główne cechy

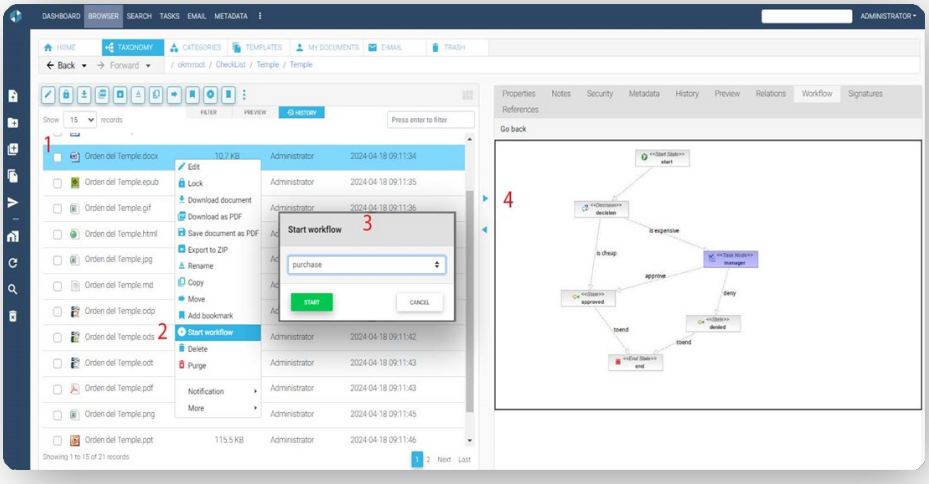
- Zaawansowane zarządzanie dokumentami oraz rozbudowane modele metadanych.
- Obsługa procesów workflow umożliwiająca modelowanie i automatyzację obiegu dokumentów.
- Mechanizmy automatycznego tagowania dokumentów oparte na regułach biznesowych.
- Wbudowany system wersjonowania dokumentów, wspierający kontrolę zmian i historię edycji.
- Możliwość podpisywania dokumentów (w tym wsparcie dla podpisów elektronicznych).
- Integracja z systemami zewnętrznymi (np. ERP, CRM).
- Rozbudowany system zarządzania użytkownikami oraz szczegółowa kontrola uprawnień.
- Dostępność w modelu chmurowym oraz on-premise.
- Wsparcie dla audytu, monitorowania oraz zgodności z wymaganiami organizacyjnymi.

4.2.2 Ograniczenia

- Przestarzałe technologie frontendowe (utrudniona rozbudowa UI).
- Wysoka złożoność architektury (trudniejsza modyfikacja i utrzymanie).
- Duże wymagania zasobowe przy skalowaniu.
- Ograniczona elastyczność w dostosowaniu UI bez ingerencji w kod.



Rys. 4.3. OpenKM - Spis dokumentów i podgląd



Rys. 4.4. OpenKM - Tworzenie nowego przepływu

4.3 M-Files

M-Files to nowoczesny system zarządzania informacją (Information Management System), który odchodzi od klasycznego podejścia DMS opartego na strukturze folderów, na rzecz modelu bazującego na metadanych oraz kontekście biznesowym. System integruje się z ekosystemem Microsoft oraz wykorzystuje mechanizmy sztucznej inteligencji do wspomagania zarządzania dokumentami, automatyzacji procesów oraz analizy treści. Dzięki zastosowaniu konfigurowalnych metadanych, workflow oraz polityk zarządzania, M-Files dostosowuje się do zmieniających się potrzeb organizacji.

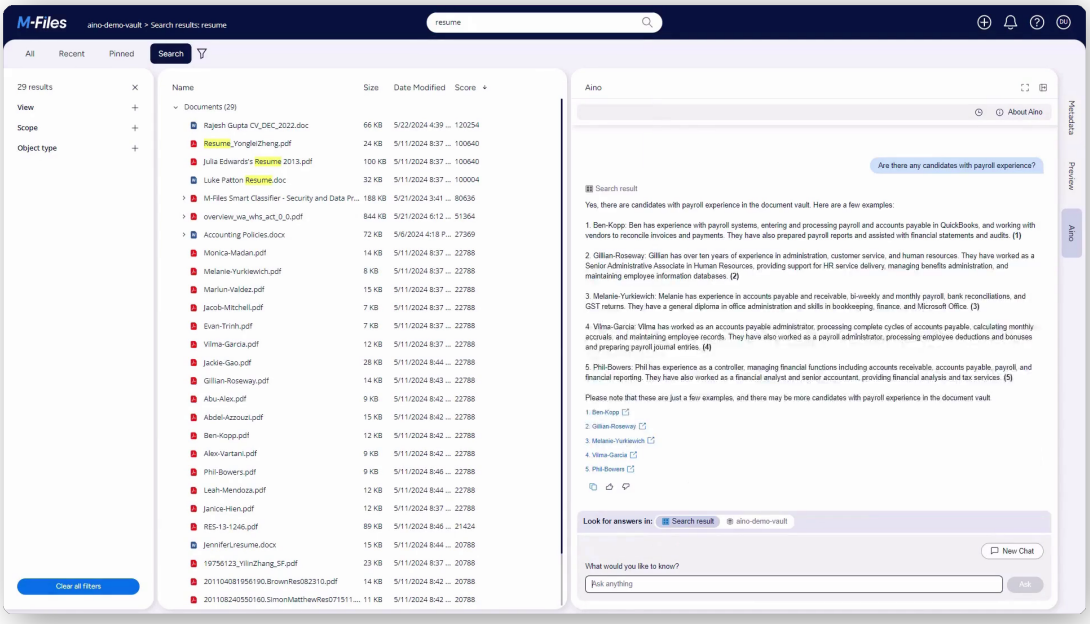
Stos technologiczny: .NET — Azure — Microsoft Graph

4.3.1 Główne cechy

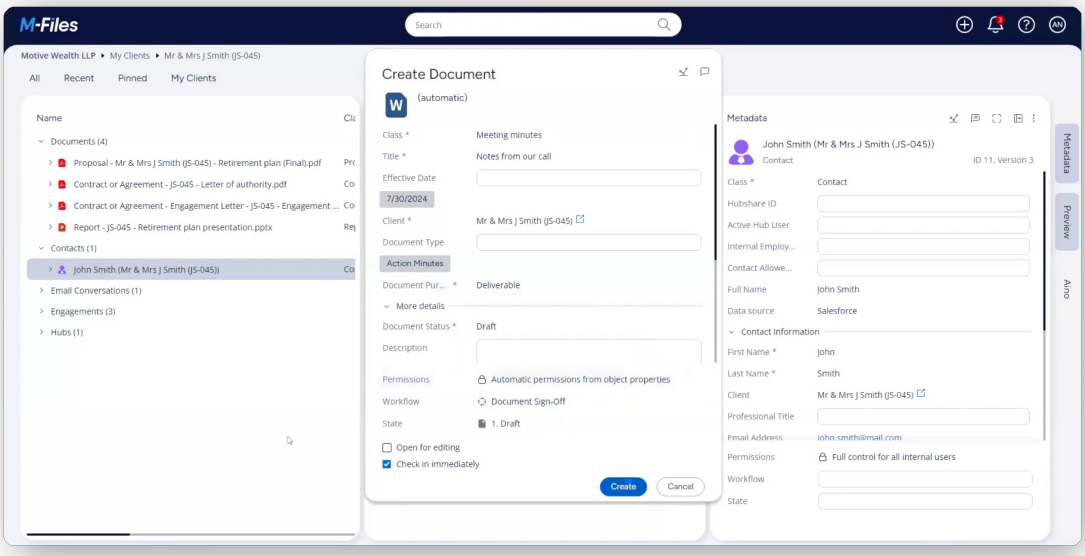
- Organizacja dokumentów oparta na metadanych, umożliwiająca powiązanie plików z kontekstem biznesowym (np. projekty, klienci, zasoby).
- Dynamiczne widoki danych, zastępujące statyczną strukturę folderów i dostosowujące się do potrzeb użytkownika.
- Zaawansowane wyszukiwanie pełnotekstowe oraz kontekstowe.
- Wbudowane mechanizmy wersjonowania dokumentów, zapewniające śledzenie zmian i historii edycji.
- Obsługa obiegów dokumentów (workflow) wraz z możliwością automatyzacji procesów biznesowych.
- Ścisła integracja z narzędziami ekosystemu Microsoft (m.in. Microsoft 365, SharePoint, Teams).
- Wsparcie dla wyszukiwania w języku naturalnym.
- Wykorzystanie mechanizmów AI do generowania podsumowań dokumentów.
- System rekomendacji sugerujący powiązane dokumenty i informacje.
- Automatyczne uzupełnianie treści oraz metadanych na podstawie analizy dokumentów.

4.3.2 Ograniczenia

- Brak dostępnej wersji open-source.
- Wysoki próg wejścia związany z koniecznością konfiguracji modeli metadanych na etapie wdrożenia.
- Silna zależność od ekosystemu Microsoft, ograniczająca niezależność technologiczną.
- Ograniczona elastyczność w środowiskach spoza infrastruktury Microsoft.



Rys. 4.5. MFiles - Spis dokumentów i wynik zapytania w języku naturalnym



Rys. 4.6. MFiles - Tworzenie nowego dokumentu

4.4 Wnioski

Rynek systemów DMS rozwija się dynamicznie, napędzany rosnącą potrzebą digitalizacji oraz automatyzacji procesów biznesowych. Analizowane rozwiązania reprezentują odmienne podejścia do projektowania systemów zarządzania dokumentami.

- Paperless-ngx odpowiada na potrzeby użytkowników indywidualnych oraz małych i średnich organizacji, oferując prostotę wdrożenia oraz podstawowe mechanizmy automatyzacji.
- OpenKM stanowi przykład rozwiązania klasy enterprise, zapewniającego rozbudowane funkcjonalności, wysoką skalowalność oraz szerokie możliwości integracji z innymi systemami.
- Z kolei M-Files prezentuje nowoczesne podejście do zarządzania informacją, w którym kluczową rolę odgrywają metadane oraz mechanizmy sztucznej inteligencji wspierające organizację i analizę dokumentów.

Na podstawie przeprowadzonej analizy można zauważyć istotną lukę rynkową. Brakuje rozwiązania, które łączyłoby prostotę i lekkość systemów takich jak Paperless-ngx z zaawansowanymi funkcjami wykorzystującymi sztuczną inteligencję, przy jednoczesnym zachowaniu niezależności od zewnętrznych dostawców usług. Istnieje zatem przestrzeń dla systemu DMS, który oferowałby nowoczesne mechanizmy AI w formie zintegrowanej, przy zachowaniu elastyczności, kontroli nad danymi oraz relatywnie niskiej złożoności wdrożenia.

5 Projekt systemu

Wymagania funkcjonalne określają funkcje i operacje, które system musi realizować, aby umożliwić użytkownikom korzystanie z jego możliwości. Opisują sposób interakcji użytkownika z aplikacją oraz działania wykonywane przez system. Stanowią podstawę do projektowania architektury i implementacji systemu. Uzupełnieniem są wymagania niefunkcjonalne, które określają parametry jakościowe, takie jak wydajność, bezpieczeństwo, użyteczność oraz kompatybilność z różnymi środowiskami.

5.1 Specyfikacja wymagań funkcjonalnych

Trywialny wymóg generowania miniaturek dokumentów został tutaj wyszczególniony ze względu na jego istotną rolę w poprawie użyteczności systemu, co jest opisywane przez S. Koyuncu Tunç [1].

5.1.1 Dodawanie zasobów

System umożliwia użytkownikowi przesyłanie plików do repozytorium systemu. Obsługiwane są formaty plików graficznych (np. JPG, PNG), plików tekstowych (TXT) oraz dokumentów PDF i DOCX. Podczas przesyłania zasobów użytkownik może dodać krótki opis dla kazdego pliku. Użytkownik może dodawać pojedyncze pliki lub wiele plików jednocześnie. System umożliwia także przesłanie archiwum ZIP, które zostanie automatycznie rozpakowane, a znajdujące się w nim pliki zostaną dodane do systemu. Pliki znajdujące się w archiwum ZIP, które nie są obsługiwane przez system, zostają automatycznie odrzucone i nie są dodawane do repozytorium. Podczas dodawania plików system wykrywa duplikaty na podstawie ich zawartości i informuje użytkownika o ich istnieniu. W przypadku wykrycia duplikatu użytkownik może zdecydować, czy plik ma zostać zapisany jako nowa wersja istniejącego dokumentu, czy jako osobny zasób.

5.1.2 Wyszukiwanie zasobów

System umożliwia wyszukiwanie dokumentów zgromadzonych w repozytorium. Użytkownik może wyszukiwać dokumenty przy użyciu zapytań w języku naturalnym, korzystając z wyszukiwania semantycznego. System pozwala także na dokładne wyszukiwanie fraz w nazwie pliku, opisie oraz w zawartości tekstowej dokumentów. Dodatkowo system umożliwia wyszukiwanie dokumentów podobnych do wskazanego pliku na podstawie jego reprezentacji wektorowej.

5.1.3 Filtrowanie i sortowanie zasobów

System umożliwia filtrowanie zasobów podczas przeglądania repozytorium. Użytkownik może zawęzić listę dokumentów na podstawie wybranych kryteriów, takich jak typ lub format pliku, przypisane kategorie, tagi oraz zakkres daty dodania. System umożliwia również sortowanie wyników przeglądania dokumentów, na według daty dodania lub innych dostępnych parametrów, co ułatwia odnajdywanie i organizowanie zasobów.

5.1.4 Zarządzanie kategoriami i tagami

System umożliwia użytkownikowi zarządzanie kategoriami oraz tagami przypisywanymi do dokumentów. Użytkownik może dodawać, edytować oraz usuwać kategorie i tagi w dedykowanym widoku zarządzania. System pozwala również na tworzenie nowych kategorii i tagów bezpośrednio podczas przypisywania ich do dokumentów. Dzięki temu użytkownik może łatwo organizować i klasyfikować zasoby w repozytorium.

5.1.5 Zarządzanie zasobami

System umożliwia użytkownikowi edycję metadanych dokumentów znajdujących się w repozytorium. Użytkownik może zmienić nazwę dokumentu, jego opis, przypisaną kategorię oraz tagi. System pozwala także na usuwanie zarówno pojedynczych dokumentów, jak i wielu dokumentów jednocześnie, w tym wszystkich dokumentów przypisanych do wybranej kategorii lub oznaczonych określonym tagiem.

5.1.6 Udostępnianie i zarządzanie udostępnieniami zasobów

System umożliwia użytkownikowi udostępnianie wybranych dokumentów innym użytkownikom systemu lub osobom zewnętrznym poprzez wygenerowanie specjalnego linku dostępowego. Użytkownik ma możliwość zablokowania pobierania udostępnionego dokumentu. Użytkownik może przeglądać listę aktywnych udostępnień dla swoich dokumentów, zmieniać ich ustawienia oraz w dowolnym momencie wycofać udostępnienie, co powoduje utratę dostępu do dokumentu przez osoby, którym został on udostępniony.

5.1.7 Kontrola wersji i wykrywanie duplikatów zasobów

System umożliwia przechowywanie i zarządzanie wersjami dokumentów. Użytkownik może przeglądać historię wersji oraz wskazać aktualnie obowiązującą wersję. System wykrywa również duplikaty przesyłanych plików na podstawie ich zawartości, informując użytkownika o istniejącej kopii. Użytkownik ma możliwość zdecydowania, czy nowy plik ma zostać zapisany jako nowa wersja istniejącego dokumentu, czy jako osobny zasób.

5.1.8 Pobieranie zasobów z opcjonalną konwersją formatu

System umożliwia użytkownikowi pobieranie zasobów, zarówno własnych, jak i udostępnionych przez innych użytkowników. Podczas pobierania użytkownik może wybrać zapis pliku w innym formacie niż oryginalny, a system automatycznie dokona konwersji pliku.

5.1.9 Generowanie miniaturek zasobów

System automatycznie generuje miniaturki dla wszystkich typów dokumentów w repozytorium, ułatwiając ich szybkie przeglądanie. Pliki graficzne są skalowane do niskiej rozdzielczości, dokumenty PDF są renderowane tak, że miniaturka przedstawia pierwszą stronę, a pliki tekstowe są konwertowane i renderowane w formie graficznej podobnie jak PDF.

5.1.10 Autoryzacja i uwierzytelnianie użytkowników

System zapewnia standardowe funkcje logowania i rejestracji użytkowników. Użytkownicy mogą tworzyć konto w systemie lub logować się przy użyciu mechanizmu jednokrotnego logowania (SSO).

System weryfikuje poprawność danych logowania i chroni przed nieautoryzowanym dostępem do zasobów cyfrowych, zapewniając bezpieczeństwo.

5.2 Specyfikacja wymagań niefunkcjonalnych

5.2.1 Wydajność (dla bazy zawierającej do 1000 dokumentów)

- Czas wyszukiwania klasycznego: poniżej 1 sekundy.
- Czas wyszukiwania semantycznego: poniżej 2 sekund.

5.2.2 Pojemność

- System musi obsługiwać minimum 1000 dokumentów.
- Maksymalny rozmiar pojedynczego pliku: 50 MB.
- Maksymalny rozmiar archiwum ZIP: 250 MB.

5.2.3 Bezpieczeństwo

- Dostęp do systemu wymaga uwierzytelnienia użytkownika.
- Użytkownik ma dostęp wyłącznie do swoich dokumentów oraz dokumentów, które zostały mu udostępnione.

5.2.4 Kompatybilność

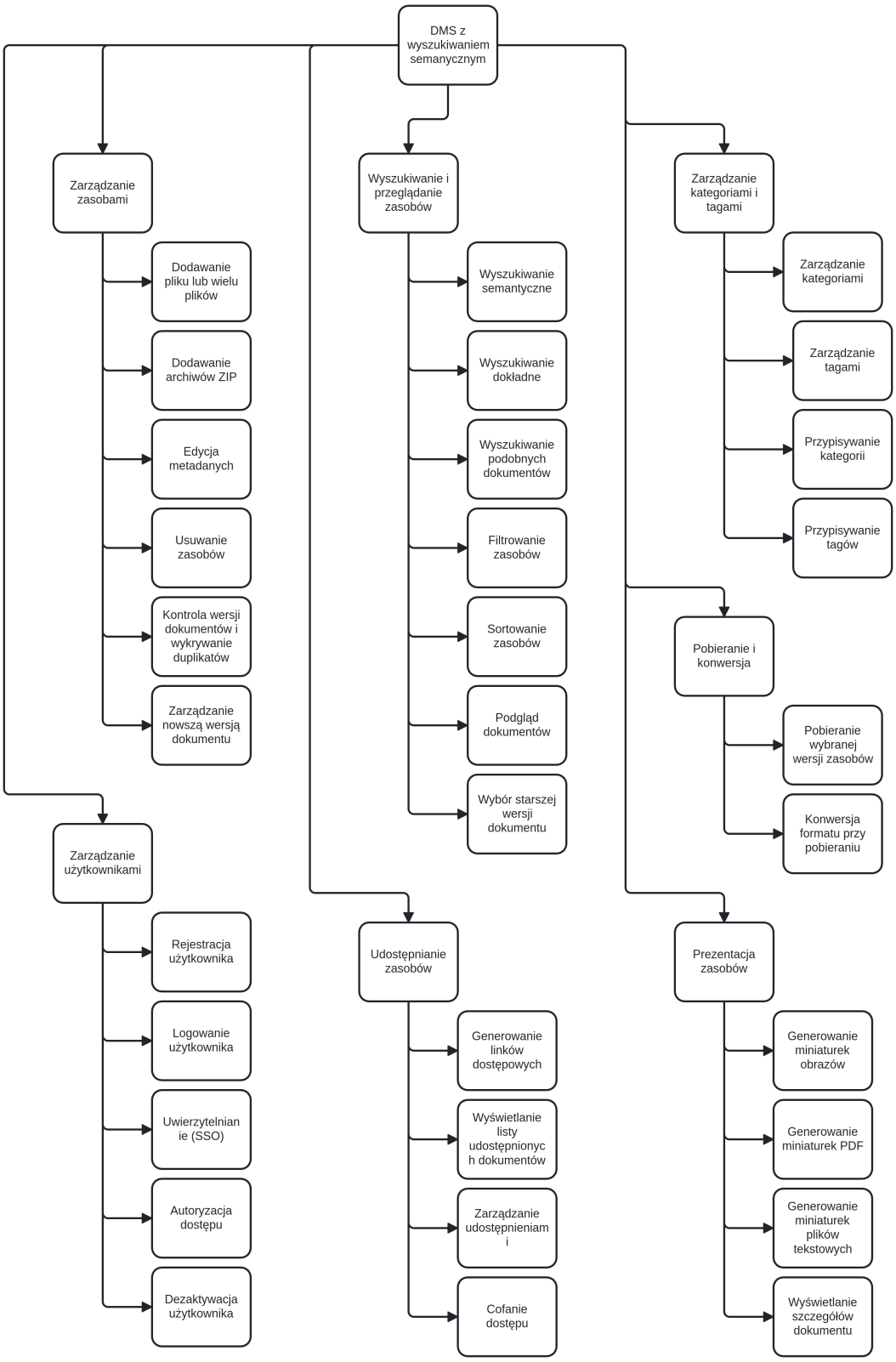
- Aplikacja internetowa musi działać poprawnie i płynnie w przeglądarkach: Firefox 120 lub nowsza, Chromium 120 lub nowsza, Edge 120 lub nowsza.

5.2.5 Użyteczność

- Interfejs użytkownika powinien umożliwiać wykonanie podstawowych operacji (np. przesyłaniedokumentów, wyszukiwanie) w maksymalnie trzech krokach.

5.3 Hierarchiczny model funkcji

Diagram hierarchii funkcji (ang. *Function Hierarchy Diagram, FHD*) przedstawia podział funkcji systemu na obszary i przypisane im funkcje szczegółowe. Kolejne poziomy hierarchii pokazują zakres poszczególnych obszarów, ułatwiając uporządkowanie wymagań funkcjonalnych.



Rys. 5.1. Diagram hierarchii funkcji systemu DMS

Na *rysunku 5.1* przedstawiono hierarchię funkcji systemu DMS z wyszukiwaniem semantycznym. Funkcję nadrzędną podzielono na siedem obszarów: zarządzanie zasobami, wyszukiwanie i przeglądanie zasobów, zarządzanie kategoriami i tagami, pobieranie i konwersję, prezentację zasobów, udostępnianie zasobów oraz zarządzanie użytkownikami.

Poszczególne obszary zostały następnie rozwinięte o bardziej szczegółowe funkcjonalności. Przykładowo moduł zarządzania zasobami obejmuje dodawanie dokumentów, edycję metadanych, kontrolę wersji oraz usuwanie zasobów. Diagram uwzględnia również funkcje związane z generowaniem miniaturek dokumentów, konwersją formatów plików, udostępnianiem zasobów za pomocą linków oraz zarządzaniem kontami użytkowników.

5.4 Historyjki użytkowników

Historyjki użytkowników (ang. *User Stories*) są jedną z technik wykorzystywanych do opisywania wymagań funkcjonalnych systemu z perspektywy jego użytkowników. Pozwalają one określić oczekiwania oraz potrzeby poszczególnych aktorów w sposób zrozumiały zarówno dla projektantów systemu, jak i jego przyszłych użytkowników. Historyjki użytkowników wspomagają proces identyfikacji funkcjonalności systemu oraz stanowią podstawę do definiowania wymagań funkcjonalnych.

W projektowanym systemie zarządzania dokumentami historyjki użytkowników zostały pogrupowane według obszarów funkcjonalnych odpowiadających najważniejszym funkcjom systemu.

5.4.1 Rejestracja i uwierzytelnianie użytkowników

1. Jako gość chcę się zarejestrować w aplikacji.
2. Jako gość chcę zalogować się do aplikacji loginem i hasłem.
3. Jako gość chcę zalogować się do aplikacji przez konto Google.

5.4.2 Dodawanie dokumentów

4. Jako użytkownik chcę przesłać jeden lub kilka plików i móc ustawić ich nazwy, kategorie i tagi w trakcie.
5. Jako użytkownik chcę przesłać wiele plików w formie archiwum ZIP.

5.4.3 Przeglądanie dokumentów

6. Jako użytkownik chcę przeglądać moje dokumenty z widocznymi miniaturkami, nazwami, tagami i kategoriami.
7. Jako użytkownik chcę kontrolować sortowanie i filtrowanie podczas przeglądania.
8. Jako użytkownik chcę wyświetlić podgląd pliku graficznego, tekstowego i PDF.

5.4.4 Wyszukiwanie dokumentów

9. Jako użytkownik chcę wyszukiwać dokumenty pełnotekstowo.
10. Jako użytkownik chcę wyszukiwać dokumenty podobne semantycznie do zapytania tekstowego.
11. Jako użytkownik chcę wyszukiwać dokumenty podobne semantycznie do wskazanego dokumentu.

5.4.5 Zarządzanie dokumentami i metadanymi

12. Jako użytkownik chcę dodać nową kategorię lub tag o określonej nazwie.
13. Jako użytkownik chcę zmienić kategorie i tagi przypisane do dokumentu.
14. Jako użytkownik chcę zmienić nazwę i opis dokumentu.
15. Jako użytkownik chcę usunąć dokument.

5.4.6 Pobieranie i konwersja dokumentów

16. Jako użytkownik chcę pobrać dokument w oryginalnej formie.
17. Jako użytkownik chcę pobrać dokument graficzny w innym formacie.
18. Jako użytkownik chcę pobrać w formacie PDF dokument, który w oryginale jest DOCX.

5.4.7 Udostępnianie dokumentów

19. Jako użytkownik chcę udostępnić dokument w formie linku.
20. Jako gość chcę zobaczyć podgląd i pobrać (z takimi samymi opcjami jak wyżej) dokument udostępniony mi przez link.
21. Jako użytkownik chcę zobaczyć status udostępnienia dokumentu.
22. Jako użytkownik chcę zobaczyć listę wszystkich udostępnionych dokumentów.
23. Jako użytkownik chcę wycofać udostępnienie dokumentu, unieważniając link.

Przedstawione historyjki użytkowników opisują najważniejsze funkcjonalności projektowanego systemu z perspektywy jego użytkowników. Stanowią one podstawę do dalszego definiowania wymagań funkcjonalnych oraz projektowania poszczególnych elementów systemu.

5.5 Scenariusze przypadków użycia

5.5.1 Pobieranie dokumentu z konwersją do formatu PDF.

Opis: Scenariusz przypadku użycia opisuje proces wyszukania dokumentu w repozytorium, wyświetlenia jego szczegółów oraz pobrania go w formacie innym niż oryginalny.

Aktorzy: Użytkownik.

Warunki początkowe: Użytkownik jest zalogowany do systemu. Dokument znajduje się w repozytorium systemu.

Warunki końcowe: Dokument zostaje pobrany przez użytkownika w formacie PDF.

Przebieg główny:

1. Użytkownik otwiera widok repozytorium dokumentów.
2. Użytkownik wyszukuje dokument przy użyciu wyszukiwania dokładnego.
3. System wyświetla listę znalezionych dokumentów.
4. Użytkownik wybiera dokument zapisany w formacie DOCX.
5. System wyświetla szczegóły dokumentu.

6. Użytkownik wybiera opcję „Pobierz”.
7. System wyświetla dostępne formaty pobierania.
8. Użytkownik wybiera format PDF.
9. System przygotowuje plik do pobrania.
10. System rozpoczyna pobieranie dokumentu.
11. Użytkownik zapisuje plik PDF na swoim urządzeniu.

Rozszerzenia:

2a. Nie znaleziono dokumentów spełniających kryteria wyszukiwania

1. System wyświetla komunikat o braku wyników.
2. Użytkownik modyfikuje zapytanie wyszukiwania.

5.5.2 Udostępnianie dokumentu po wyszukaniu semantycznym

Opis: Scenariusz przypadku użycia opisuje proces odnalezienia dokumentu przy użyciu wyszukiwania semantycznego oraz udostępnienia go innemu użytkownikowi poprzez wygenerowanie linku dostępowego.

Aktorzy: Użytkownik.

Warunki początkowe: Użytkownik jest zalogowany do systemu. Dokument znajduje się w repozytorium.

Warunki końcowe: Wygenerowany zostaje aktywny link umożliwiający dostęp do dokumentu.

Przebieg główny:

1. Użytkownik otwiera widok repozytorium dokumentów.
2. Użytkownik wprowadza zapytanie w języku naturalnym.
3. System wykonuje wyszukiwanie semantyczne.
4. System wyświetla listę dokumentów najbardziej podobnych do zapytania.
5. Użytkownik wybiera dokument z listy wyników.
6. System wyświetla szczegóły dokumentu.
7. Użytkownik wybiera opcję „Udostępnij”.
8. System generuje unikalny link dostępu do dokumentu.
9. System zapisuje informacje o udostępnieniu.
10. System wyświetla wygenerowany link.
11. Użytkownik przekazuje link wybranej osobie.

Rozszerzenia:

4a. Nie znaleziono dokumentów podobnych do zapytania

1. System wyświetla komunikat o braku wyników.
2. Użytkownik modyfikuje treść zapytania.

8a. Dokument jest już udostępniony

1. System informuje użytkownika o istniejącym aktywnym udostępnieniu.
2. System wyświetla wcześniej wygenerowany link.

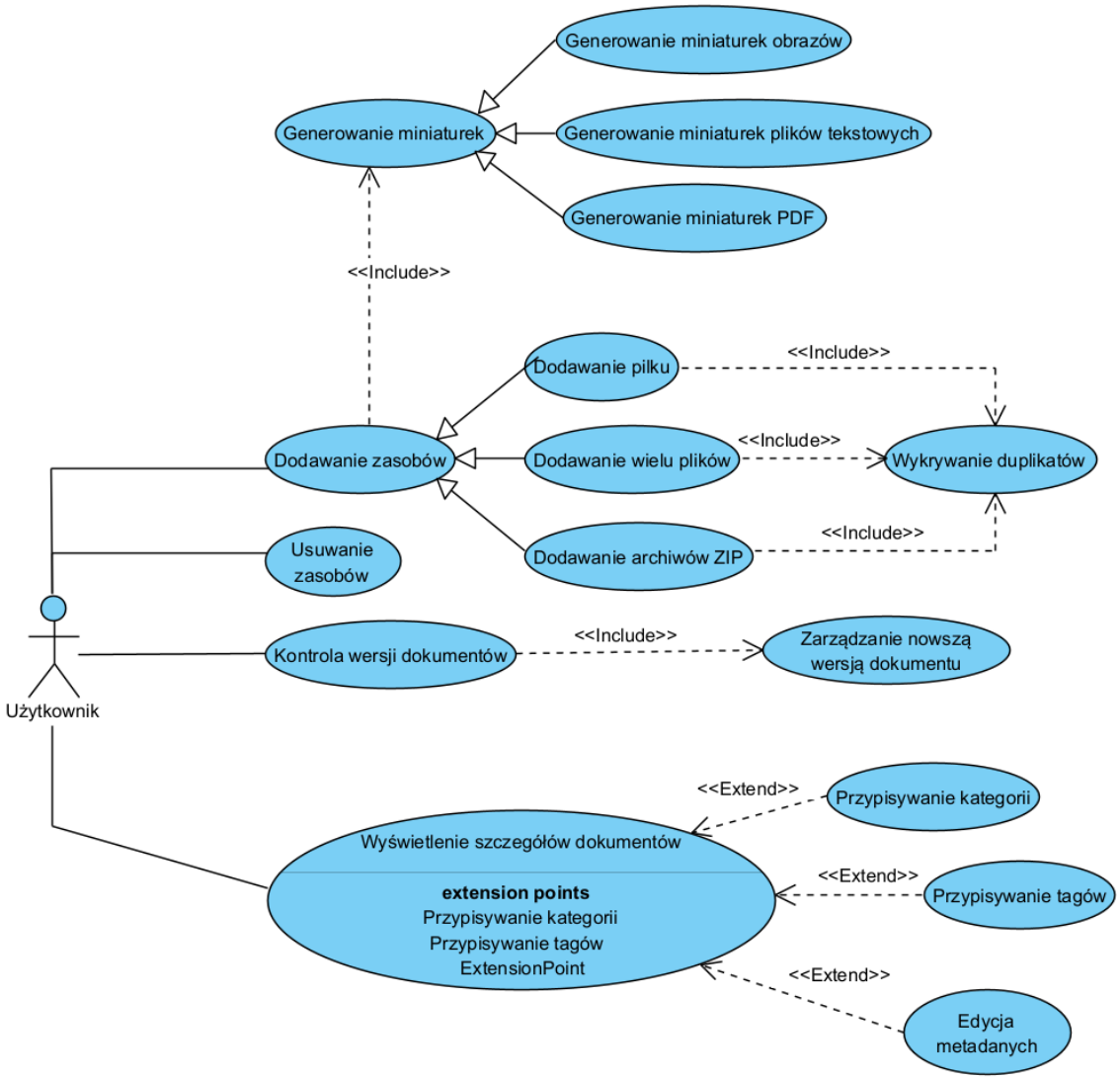
11a. Użytkownik chce wycofać udostępnienie

1. Użytkownik otwiera listę aktywnych udostępnień.
2. Użytkownik wybiera opcję „Cofnij dostęp”.
3. System unieważnia link dostępu.
4. System zapisuje zmianę statusu udostępnienia.

5.6 Diagramy przypadków użycia

Diagramy przypadków użycia (ang. *Use Case Diagrams*) w języku UML służą do modelowania funkcjonalności systemu z perspektywy jego użytkowników (aktorów). Pozwalają one określić zakres projektowanego rozwiązania oraz operacje dostępne dla poszczególnych grup użytkowników. W systemie DMS funkcjonalności te zostały podzielone na powiązane obszary tematyczne na podstawie zdefiniowanych wymagań funkcjonalnych.

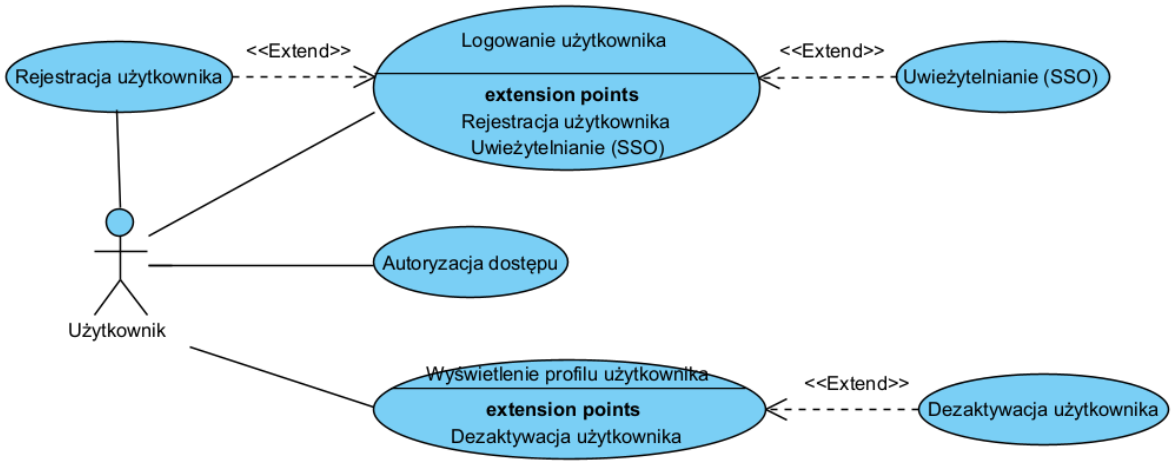
Poniżej przedstawiono diagramy przypadków użycia dla najważniejszych obszarów funkcjonalnych systemu.



Rys. 5.2. Diagram przypadków użycia: zarządzanie zasobami

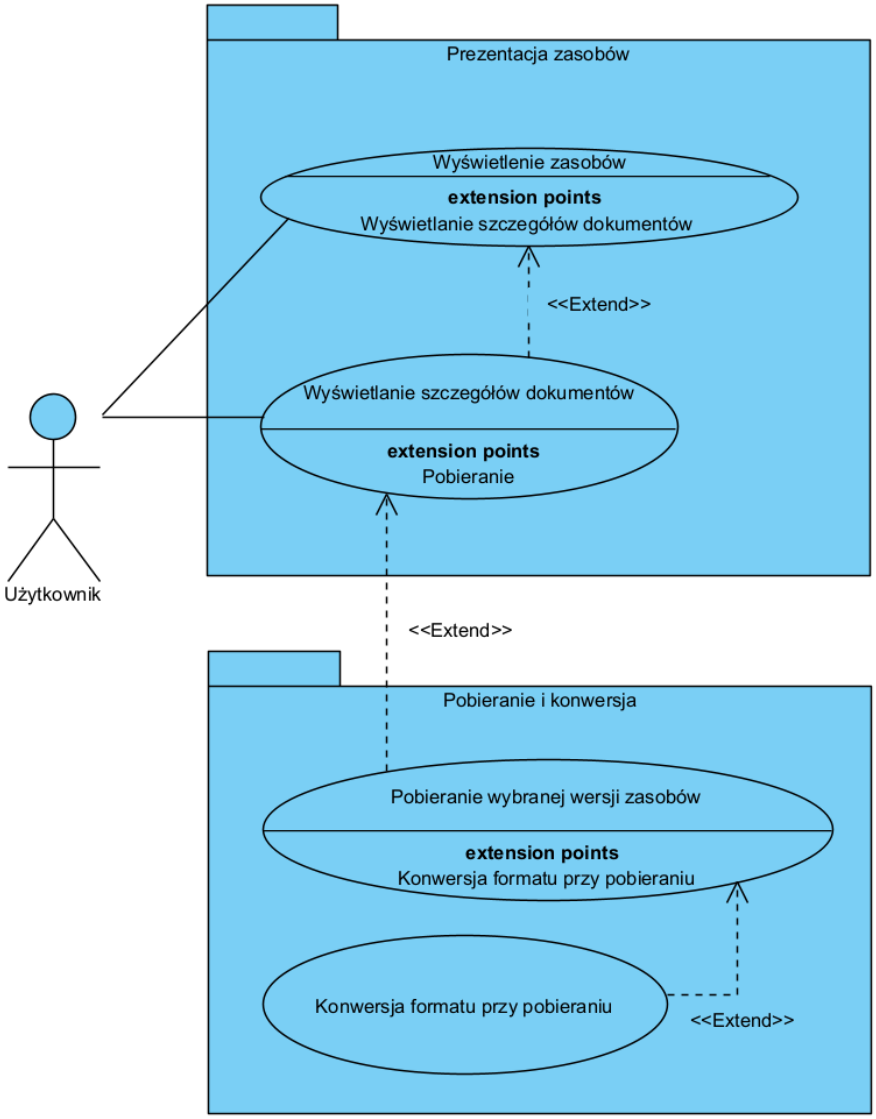
Na *rysunku 5.2* przedstawiono przypadki użycia dotyczące zarządzania dokumentami. Dodawanie zasobów ma trzy warianty: dodawanie pojedynczego pliku, wielu plików oraz archiwum ZIP. Każdy wariant obejmuje wykrywanie duplikatów, a dodawanie zasobów uwzględnia również generowanie miniaturek obrazów, plików tekstowych i dokumentów PDF. Użytkownik może usuwać zasoby oraz korzystać z kontroli wersji obejmującej zarządzanie nowszą wersją dokumentu. Wyświetlenie szczegółów dokumentu jest rozszerzane o przypisanie kategorii, przypisanie tagów lub edycję metadanych.

Mam pisać ‘może być rozszerzone‘ / ‘jest rozszerzone‘ / ‘jest rozszerzane’



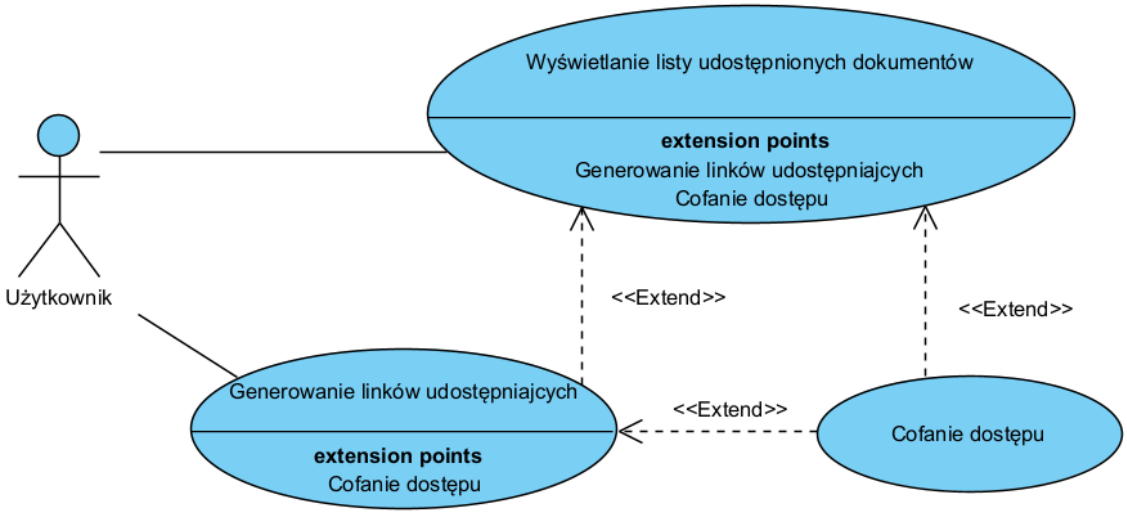
Rys. 5.3. Diagram przypadków użycia: obsługa konta użytkownika

Na *rysunku 5.3* przedstawiono przypadki użycia związane z obsługą konta użytkownika. Logowanie może zostać rozszerzone o rejestrację konta lub uwierzytelnianie z wykorzystaniem mechanizmu jednokrotnego logowania (SSO). Diagram uwzględnia także autoryzację, czyli kontrolę uprawnień dostępu do zasobów. Użytkownik może wyświetlić swój profil i z tego poziomu zainicjować dezaktywację konta.



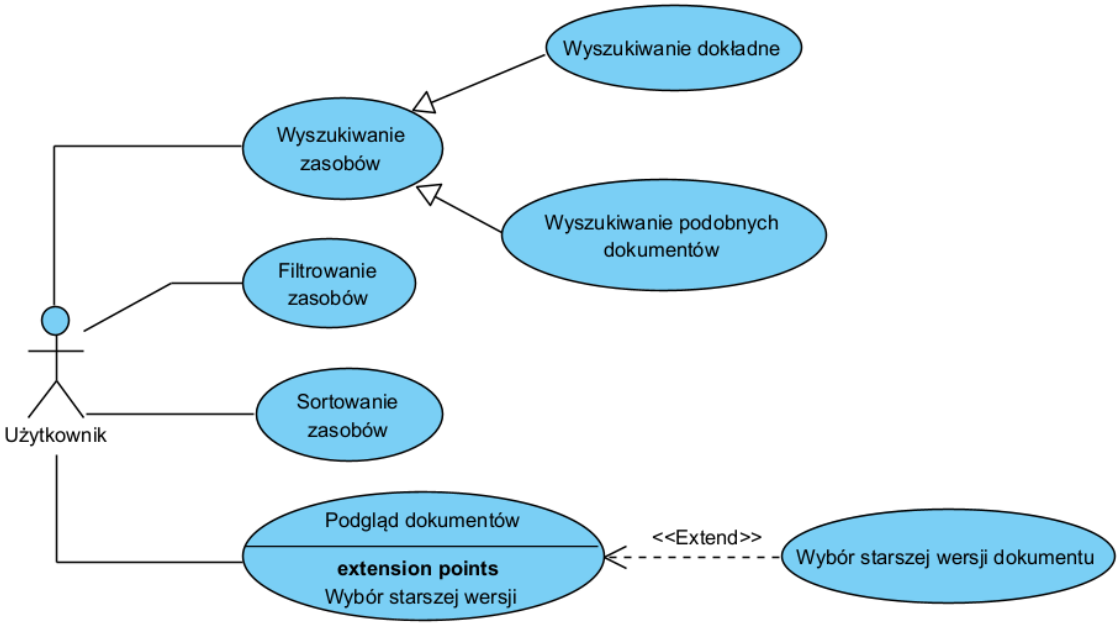
Rys. 5.4. Diagram przypadków użycia: prezentacja i pobieranie zasobów

Na *rysunku 5.4* przedstawiono powiązania między prezentacją zasobów a ich pobieraniem. Wyświetlanie zasobów może zostać rozszerzone o wyświetlenie szczegółów dokumentu, a widok szczegółów umożliwia pobranie wybranej wersji zasobu. Opcjonalnym rozszerzeniem pobierania jest konwersja dokumentu do innego formatu. Podział na dwa pakiety wyróżnia funkcje prezentacji oraz pobierania i konwersji.



Rys. 5.5. Diagram przypadków użycia: udostępnianie zasobów

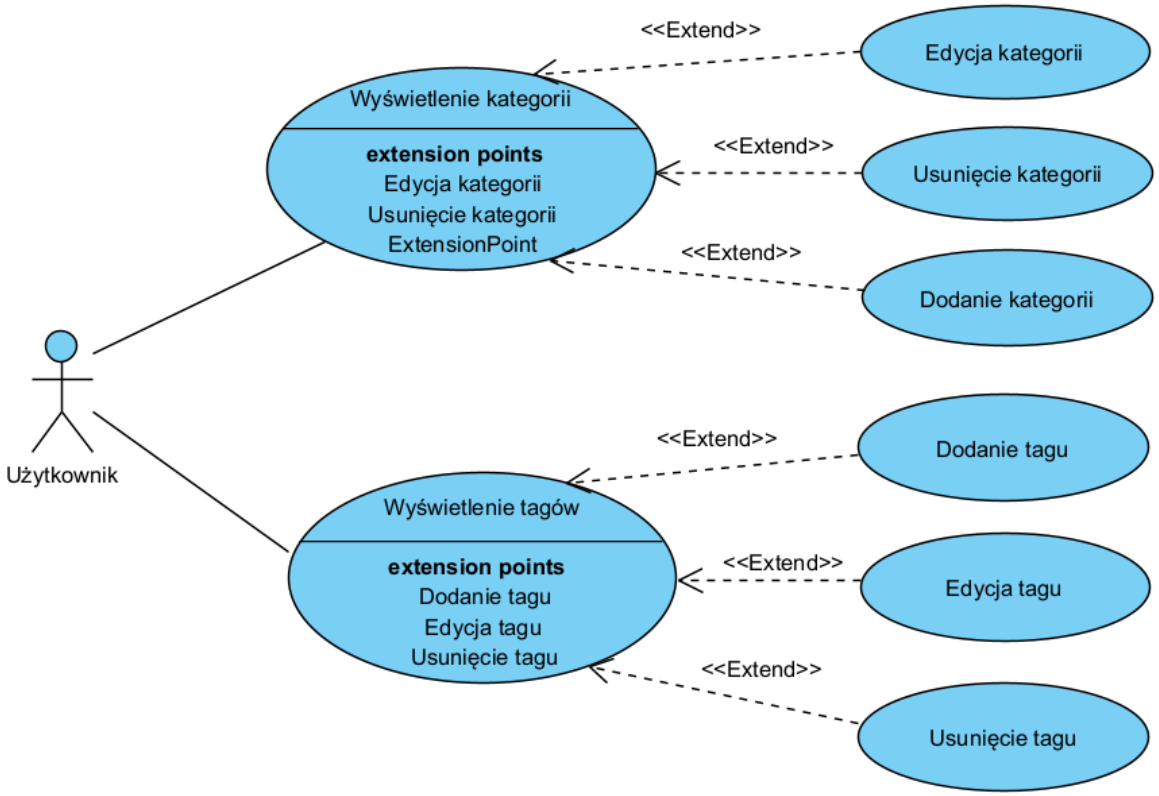
Na *rysunku 5.5* przedstawiono przypadki użycia dotyczące udostępniania dokumentów. Użytkownik może wyświetlić listę udostępnionych dokumentów oraz wygenerować link udostępniający. Przeglądanie listy może zostać rozszerzone o wygenerowanie linku lub cofnięcie dostępu. Cofnięcie dostępu stanowi również rozszerzenie przypadku generowania linku i pozwala wycofać utworzone udostępnienie.



Rys. 5.6. Diagram przypadków użycia: wyszukiwanie i przeglądanie zasobów

Brakuje wyszukiwania semantycznego na diag

Na *rysunku 5.6* przedstawiono przypadki użycia związane z odnajdywaniem i przeglądaniem dokumentów. Wyszukiwanie zasobów ma trzy warianty: wyszukiwanie dokładne, wyszukiwanie semantyczne oraz wyszukiwanie podobnych dokumentów. Użytkownik może ponadto filtrować i sortować zasoby oraz wyświetlać ich podgląd. Rozszerzeniem podglądu jest wybór starszej wersji dokumentu, umożliwiający przeglądanie jego wcześniejszej zawartości.



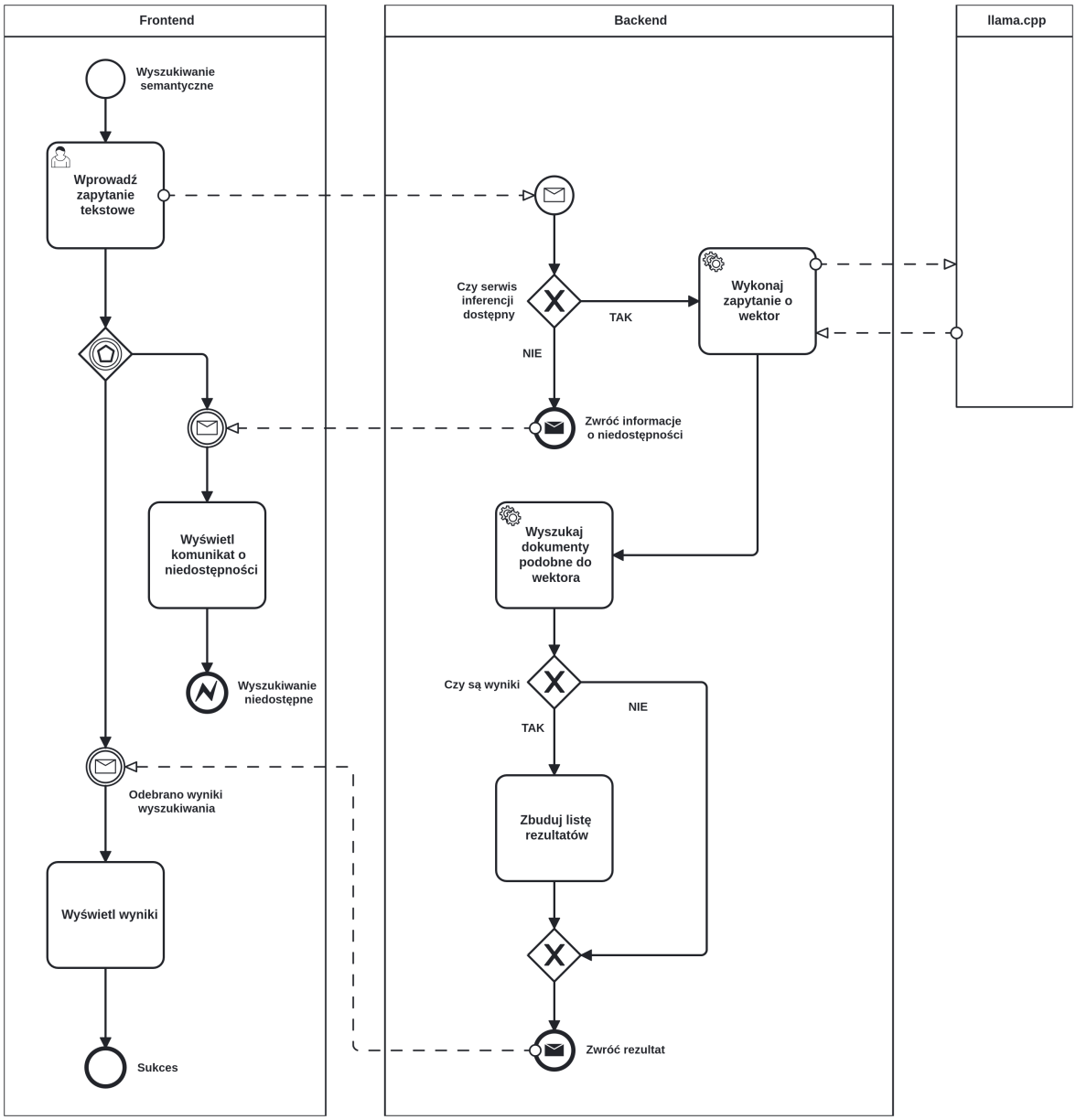
Rys. 5.7. Diagram przypadków użycia: zarządzanie kategoriami i tagami

Na *rysunku 5.7* pokazano przypadki użycia związane z zarządzaniem kategoriami i tagami. Użytkownik może wyświetlać kategorie oraz tagi. Każdy z tych przypadków może zostać rozszerzony o dodanie, edycję lub usunięcie odpowiedniego elementu. Diagram opisuje utrzymywanie słowników służących do organizacji dokumentów; przypisywanie kategorii i tagów do dokumentu pokazano na diagramie zarządzania zasobami.

5.7 Diagramy procesów biznesowych

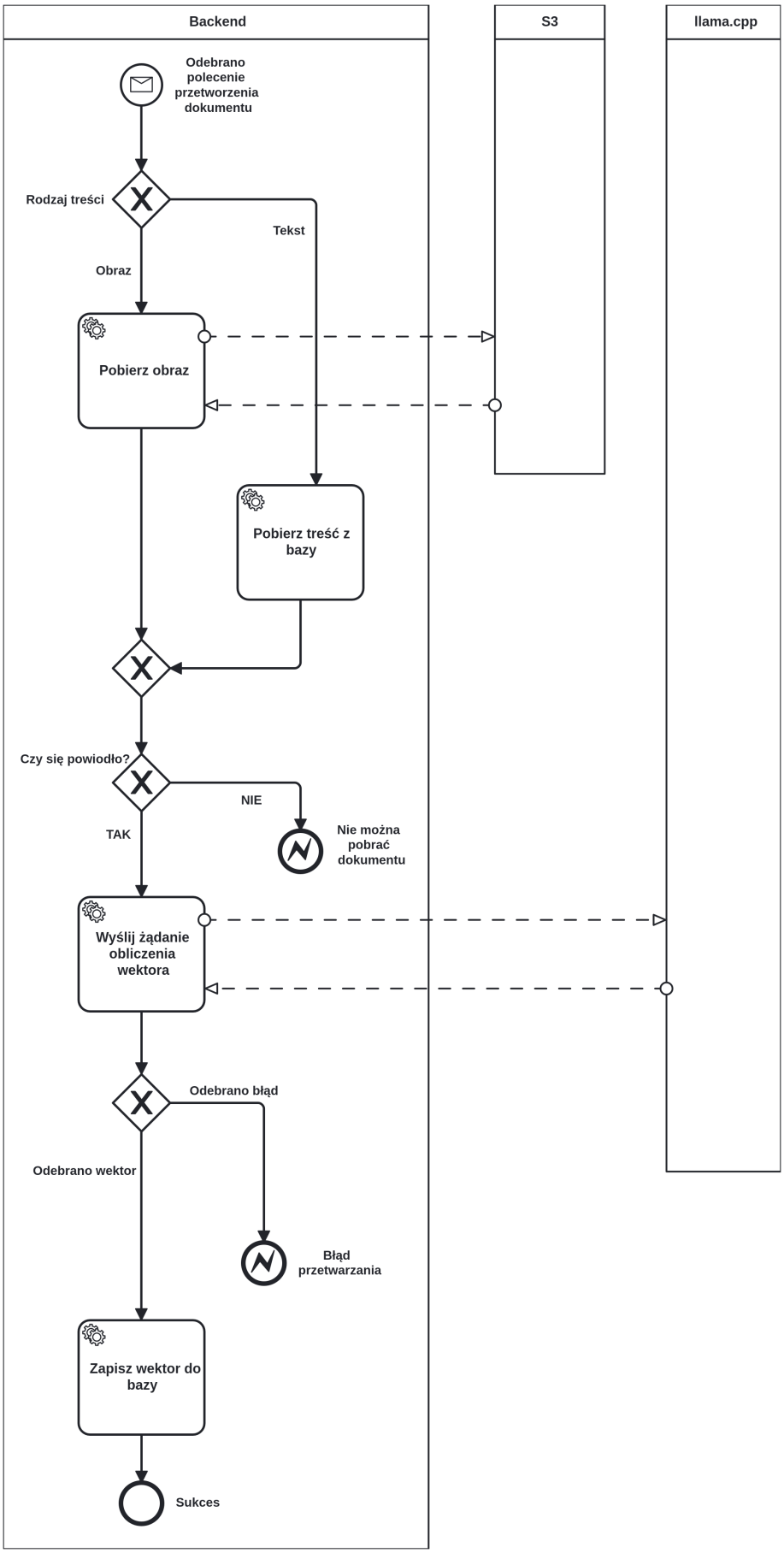
Diagramy wykonane w notacji BPMN (ang. *Business Process Model and Notation*) służą do graficznego przedstawienia sekwencji zadań i przepływu informacji w ramach realizowanych procesów. Pozwalają one na precyzyjne odzwierciedlenie interakcji pomiędzy warstwą wizualną aplikacji, logiką biznesową a zewnętrznymi modułami i usługami składowymi. W prezentowanych modelach wyróżniono uczestników odpowiedzialnych za interfejs użytkownika (Frontend), logikę aplikacji (Backend), przechowywanie plików (S3) oraz obliczanie reprezentacji wektorowych (llama.cpp). Przepływy komunikatów pokazują wymianę informacji między uczestnikami.

Poniżej opisano pięć procesów związanych z obsługą dokumentów i wyszukiwaniem semantycznym.



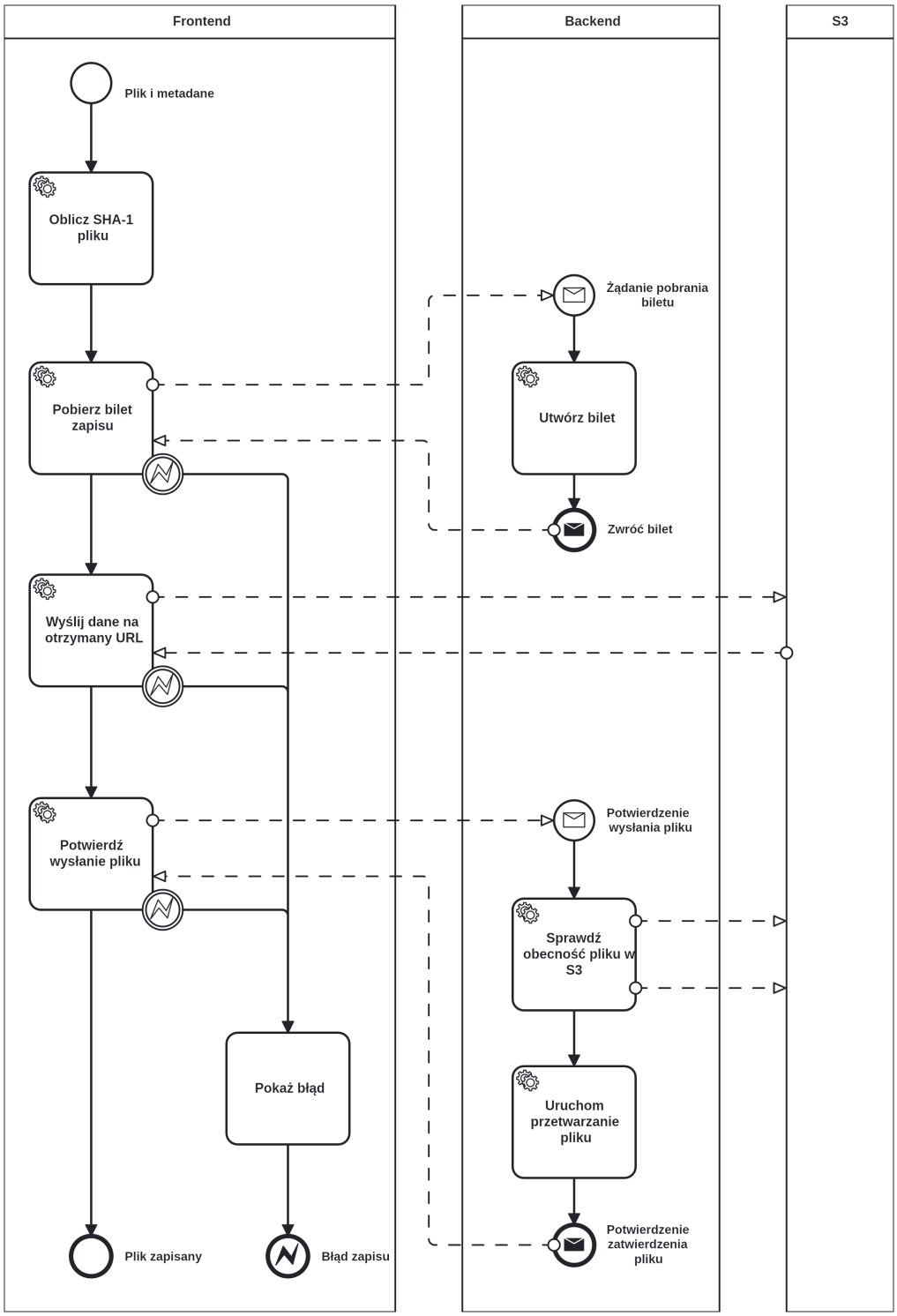
Rys. 5.8. Diagram BPMN procesu wyszukiwania semantycznego

Na *rysunku 5.8* przedstawiono proces wyszukiwania semantycznego. Po wprowadzeniu zapytania przez użytkownika interfejs przesyła je do backendu, który sprawdza dostępność serwisu inferencji. Niedostępność serwisu powoduje zwrócenie informacji do interfejsu, wyświetlenie komunikatu i zakończenie procesu. Jeżeli serwis jest dostępny, backend zleca obliczenie wektora zapytania usłudze llama.cpp, a następnie wyszukuje dokumenty na podstawie podobieństwa do otrzymanego wektora. W przypadku znalezienia dokumentów buduje listę wyników. Proces kończy się wyświetleniem wyników użytkownikowi.



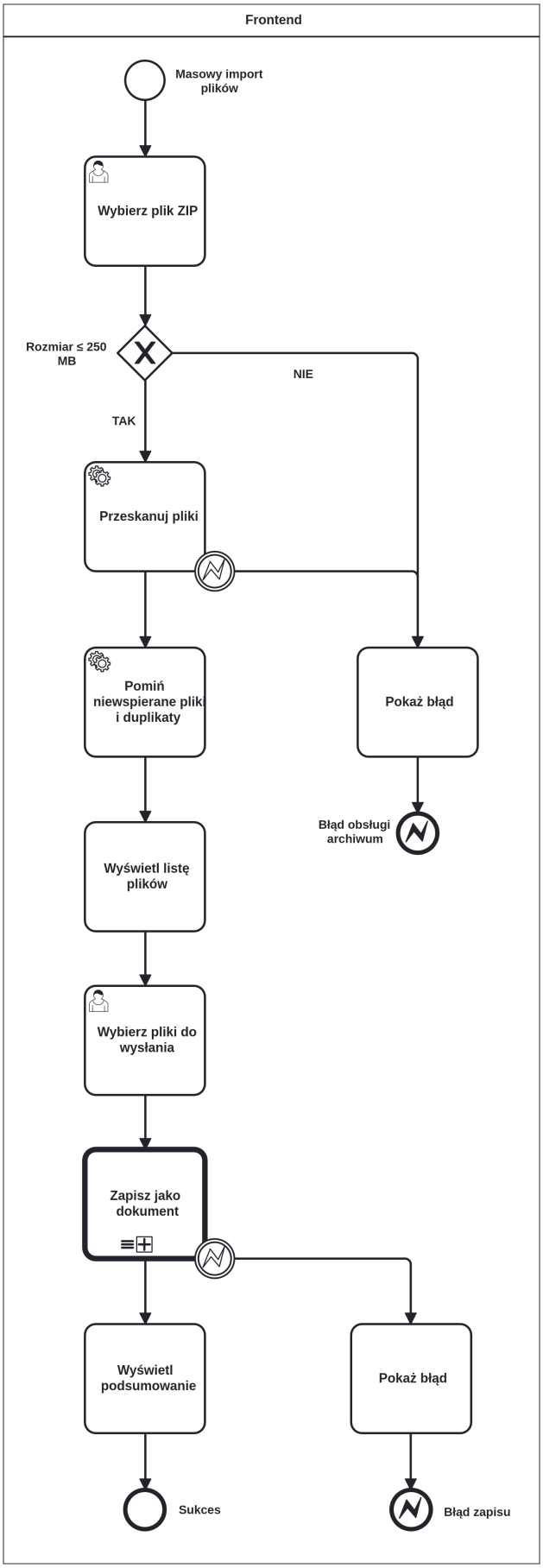
Rys. 5.9. Diagram BPMN procesu generowania wektora osadzenia

Na *rysunku 5.9* przedstawiono proces generowania wektora osadzenia, czyli reprezentacji semantycznej zawartości dokumentu. Po odebraniu polecenia przetworzenia dokumentu z kolejki zadań wątek przetwarzania wybiera sposób pobrania danych zależnie od rodzaju treści: obraz pobiera z magazynu S3, a tekst z bazy danych. Niepowodzenie pobrania danych kończy proces błędem. Po uzyskaniu zawartości backend wysyła do usługi llama.cpp żądanie obliczenia wektora. Otrzymany wektor zapisuje w bazie danych, natomiast błąd zwrócony przez usługę powoduje zakończenie procesu błędem przetwarzania.



Rys. 5.10. Diagram BPMN procesu zapisu dokumentu

Na *rysunku 5.10* przedstawiono proces zapisu dokumentu z wykorzystaniem biletu zapisu. Interfejs oblicza skrót SHA-1 pliku, a następnie zwraca się do backendu o bilet zawierający podpisany kryptograficznie adres URL do przesłania danych. Po otrzymaniu biletu interfejs wysyła plik bezpośrednio do magazynu S3 i potwierdza zakończenie wysyłania w backendzie. Backend sprawdza obecność pliku w magazynie, uruchamia jego przetwarzanie i zwraca potwierdzenie zatwierdzenia dokumentu. Błędy pobrania biletu, wysyłania danych lub potwierdzenia zapisu prowadzą do wyświetlenia komunikatu i zakończenia procesu błędem zapisu.

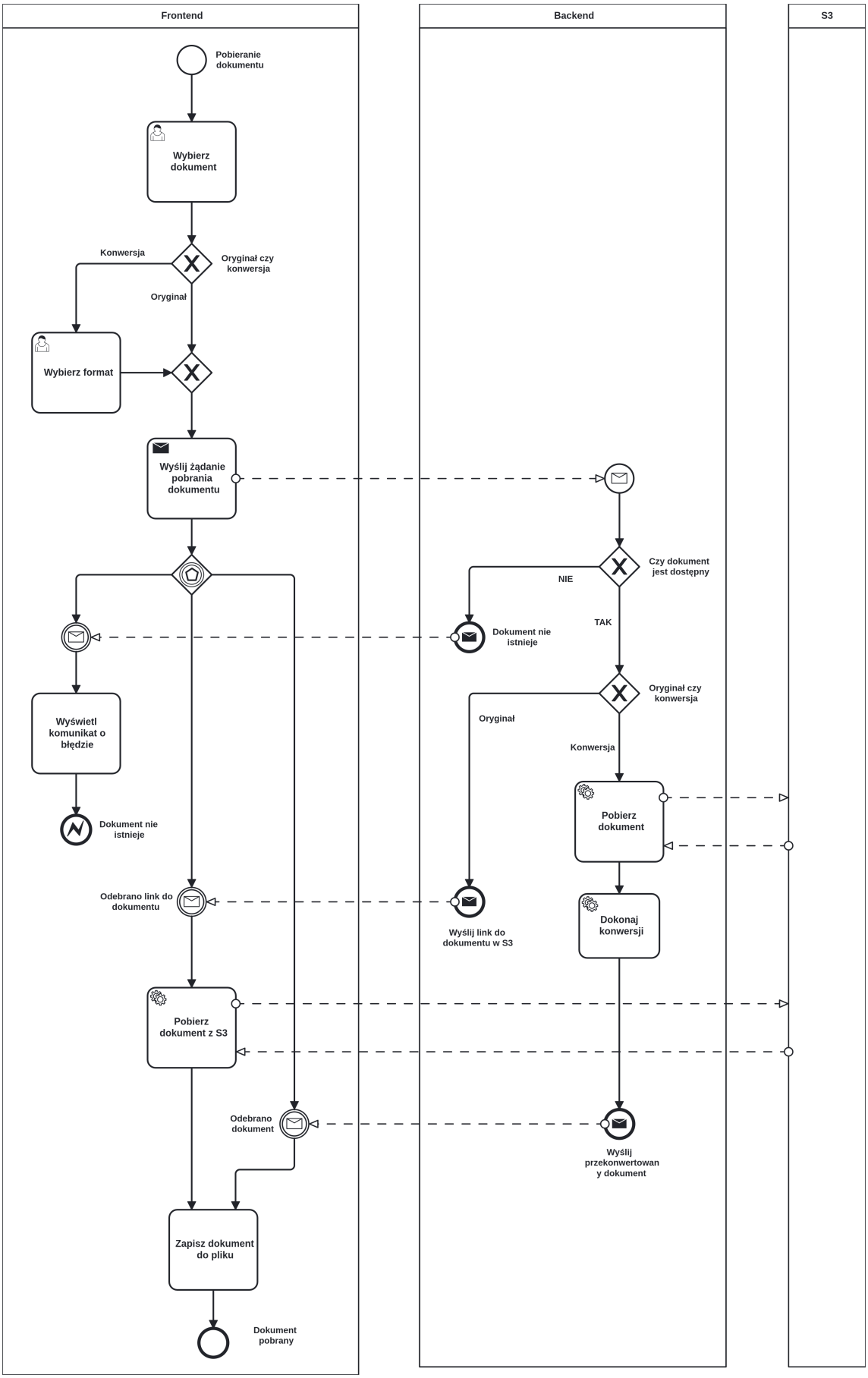


Rys. 5.11. Diagram BPMN procesu importu dokumentów z archiwum ZIP

Deduplikacja i zapis dokumentu komunikują się z backendem - nwm czy go tu pokazać, zapis dokumentu to wywołaniu procesu wyżej więc raczej nie potrzebuje

Ogólnie z tym SHA1 to średnio wygląda bo on jest liczony przed podjęciem decyzji o zapisie(właśnie w celu dedup). Ale poprawne diag byłyby za długie

Na *rysunku 5.11* przedstawiono proces importu dokumentów z archiwum ZIP. Użytkownik wybiera archiwum w interfejsie, który sprawdza, czy jego rozmiar nie przekracza 250 MB. Po spełnieniu tego warunku następuje skanowanie zawartości oraz pominięcie nieobsługiwanych plików i duplikatów. Interfejs wyświetla listę pozostałych plików, a użytkownik wybiera dokumenty do wysłania. Dla każdego wybranego pliku wykonywany jest proces zapisu dokumentu. Po zakończeniu importu wyświetlane jest podsumowanie. Przekroczenie limitu rozmiaru lub błąd skanowania prowadzi do zakończenia procesu błędem obsługi archiwum, natomiast niepowodzenie zapisu dokumentu prowadzi do zakończenia błędem zapisu.

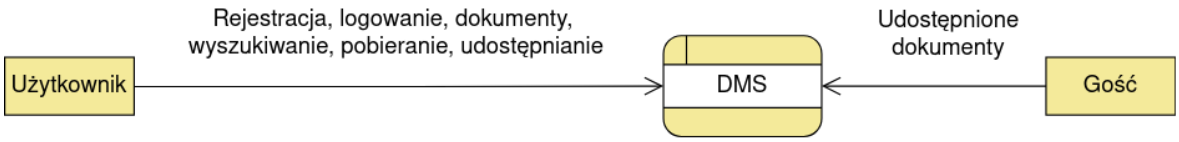


Rys. 5.12. Diagram BPMN procesu pobierania i konwersji dokumentu

Na *rysunku 5.12* przedstawiono proces pobierania dokumentu w oryginalnym formacie lub po konwersji. Użytkownik wybiera dokument, a w przypadku konwersji również format docelowy. Interfejs wysyła żądanie do backendu, który sprawdza dostępność dokumentu. Jeżeli dokument jest niedostępny, interfejs wyświetla komunikat o błędzie i kończy proces. Dla oryginalnego pliku backend zwraca link umożliwiający pobranie danych bezpośrednio z S3. W wariancie z konwersją backend pobiera plik z magazynu, przekształca go do wybranego formatu i przesyła wynik do interfejsu. Obie poprawne ścieżki kończą się zapisaniem dokumentu do pliku po stronie użytkownika.

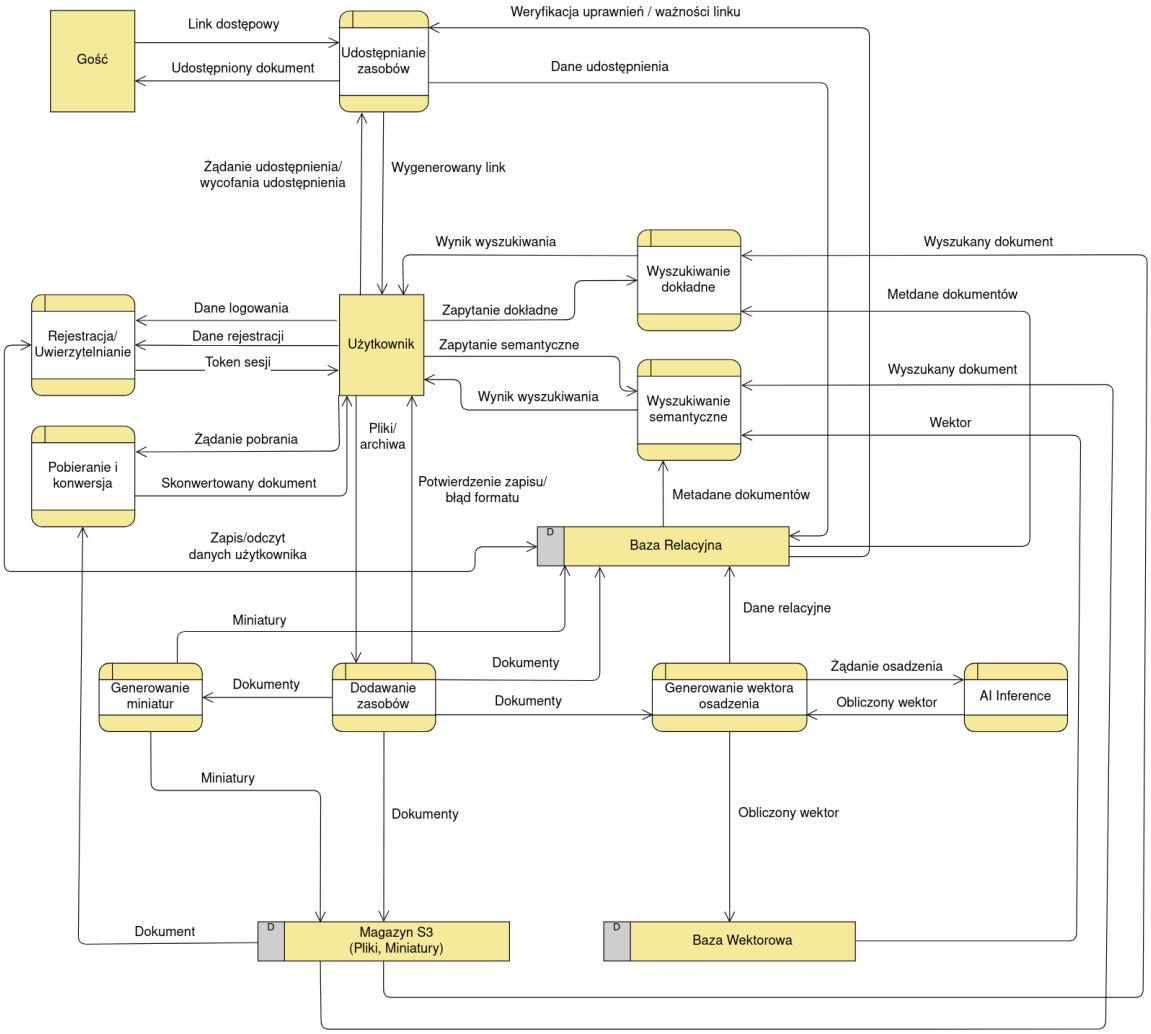
5.8 Diagramy przepływu danych

Diagramy przepływu danych (ang. *Data Flow Diagram, DFD*) przedstawiają obieg informacji między aktorami, procesami systemu oraz magazynami danych. Diagram kontekstowy pokazuje system jako pojedynczy proces, natomiast poziom 1 przedstawia jego podział na procesy szczegółowe.



Rys. 5.13. Diagram kontekstowy systemu DMS (DFD, poziom 0)

Na *rysunku 5.13* przedstawiono diagram kontekstowy systemu DMS, oznaczony jako poziom 0. Wyróżniono dwóch aktorów: użytkownika oraz gościa. Zakres kontaktu użytkownika z systemem obejmuje rejestrację, logowanie, obsługę dokumentów, wyszukiwanie, pobieranie i udostępnianie zasobów. Udział gościa ogranicza się do korzystania z udostępnionych dokumentów. Szczegółowe kierunki przepływu danych dla tych operacji przedstawiono na poziomie 1.



Rys. 5.14. Diagram przepływu danych systemu DMS (DFD, poziom 1)

Zmienić AI Inference na llama.cpp

Na *rysunku 5.14* przedstawiono dekompozycję systemu DMS na procesy przetwarzające dane. Pokazano przepływy danych łączące procesy aplikacji z użytkownikiem, gościem oraz usługą llama.cpp. Wyróżniono trzy logiczne magazyny danych: bazę relacyjną przechowującą dane użytkowników, dokumentów i udostępnień, bazę wektorową przechowującą wektory osadzeń oraz magazyn S3 zawierający pliki i miniatury.

Podział na bazę relacyjną i wektorową ma charakter logiczny i służy wyróżnieniu ich ról na diagramie. W projekcie obie funkcje realizuje jedna instancja PostgreSQL z rozszerzeniem *pgvector*. Pliki dokumentów i miniatury są przechowywane oddzielnie w magazynie obiektowym S3.

Na poziomie 1 wyróżniono następujące procesy:

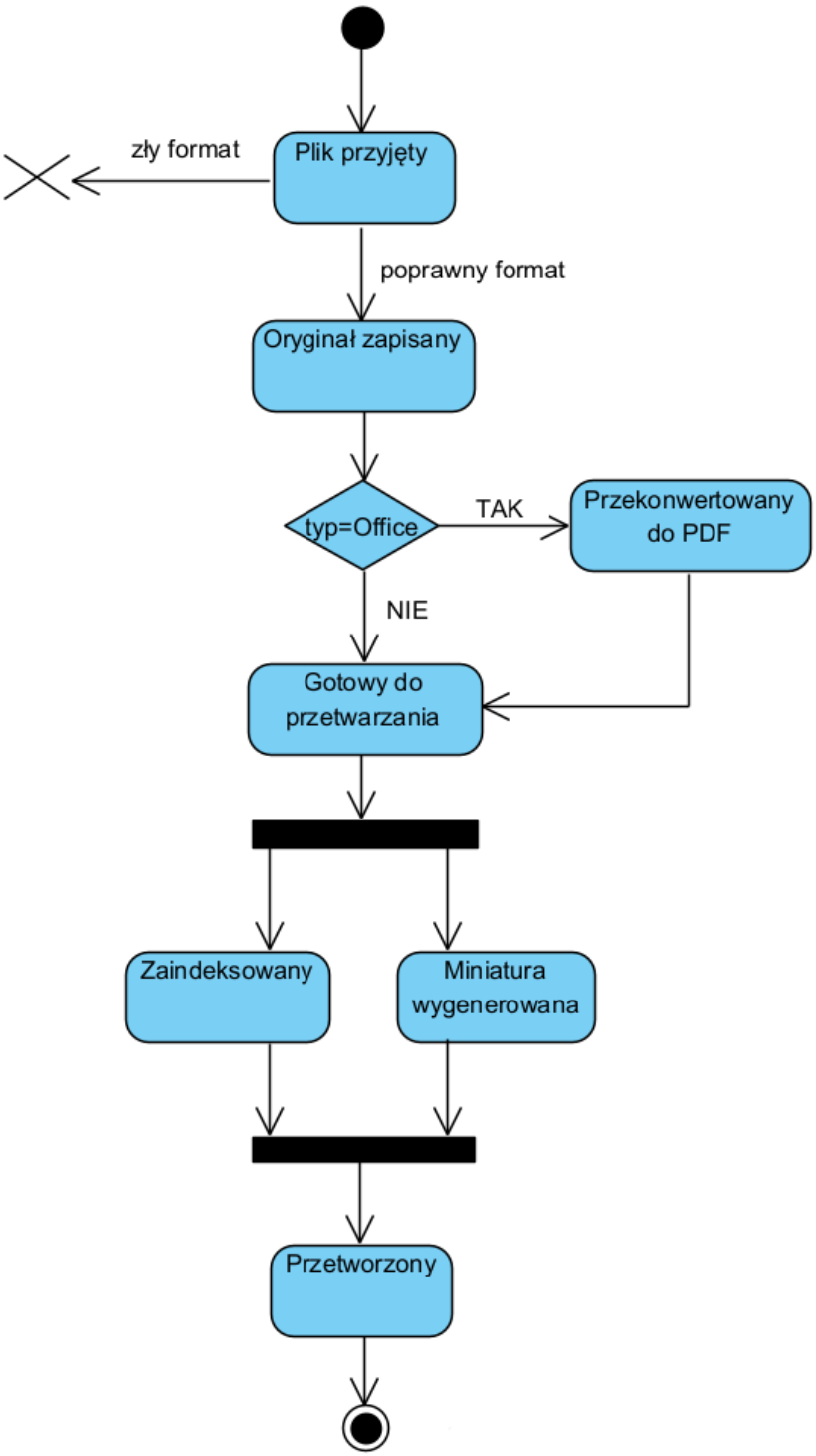
- **Rejestracja i uwierzytelnianie**: przetwarzanie danych rejestracji i logowania, odczyt i zapis danych użytkownika w bazie relacyjnej oraz zwracanie tokenu sesji.
- **Dodawanie zasobów**: przyjmowanie plików lub archiwów, zapis danych dokumentów w bazie relacyjnej i plików w S3 oraz przekazywanie dokumentów do generowania miniatur i wektorów osadzeń. Użytkownik otrzymuje potwierdzenie zapisu lub informację o błędzie formatu.
- **Udostępnianie zasobów**: obsługa żądań utworzenia i wycofania udostępnienia oraz zwracanie

wygenerowanego linku. Dostęp gościa wymaga przekazania linku i weryfikacji uprawnień lub ważności udostępnienia na podstawie danych z bazy relacyjnej.

- **Pobieranie i konwersja**: pobieranie dokumentu z magazynu S3 i przekazywanie go użytkownikowi, z możliwością konwersji formatu.
- **Wyszukiwanie dokładne**: przetwarzanie zapytania użytkownika z wykorzystaniem metadanych z bazy relacyjnej i dokumentów z magazynu S3 oraz zwracanie wyniku wyszukiwania.
- **Wyszukiwanie semantyczne**: przetwarzanie zapytania z wykorzystaniem metadanych, wektorów osadzeń i dokumentów oraz przekazywanie wyników użytkownikowi.
- **Generowanie miniatur**: przygotowywanie miniatur dokumentów przekazanych przez proces dodawania zasobów, zapis miniatur w S3 oraz powiązanych danych w bazie relacyjnej.
- **Generowanie wektora osadzenia**: zlecanie obliczeń usłudze llama.cpp, odbieranie wektora i zapisywanie go w bazie wektorowej oraz zapisywanie danych powiązanych w bazie relacyjnej.

5.9 Diagram stanów

Diagram stanów (ang. *State Machine Diagram*) przedstawia etapy przetwarzania dokumentu oraz warunki przejść między nimi. Prezentowany model obejmuje przyjęcie pliku, zapis oryginału, ewentualną konwersję oraz przygotowanie danych potrzebnych do wyszukiwania i prezentacji dokumentu.



Rys. 5.15. Diagram stanów dokumentu

Na *rysunku 5.15* przedstawiono przebieg przetwarzania dokumentu od stanu *Plik przyjęty*. Poprawny format pozwala przejść do stanu *Oryginał zapisany*, natomiast niepoprawny format powoduje przerwanie przetwarzania.

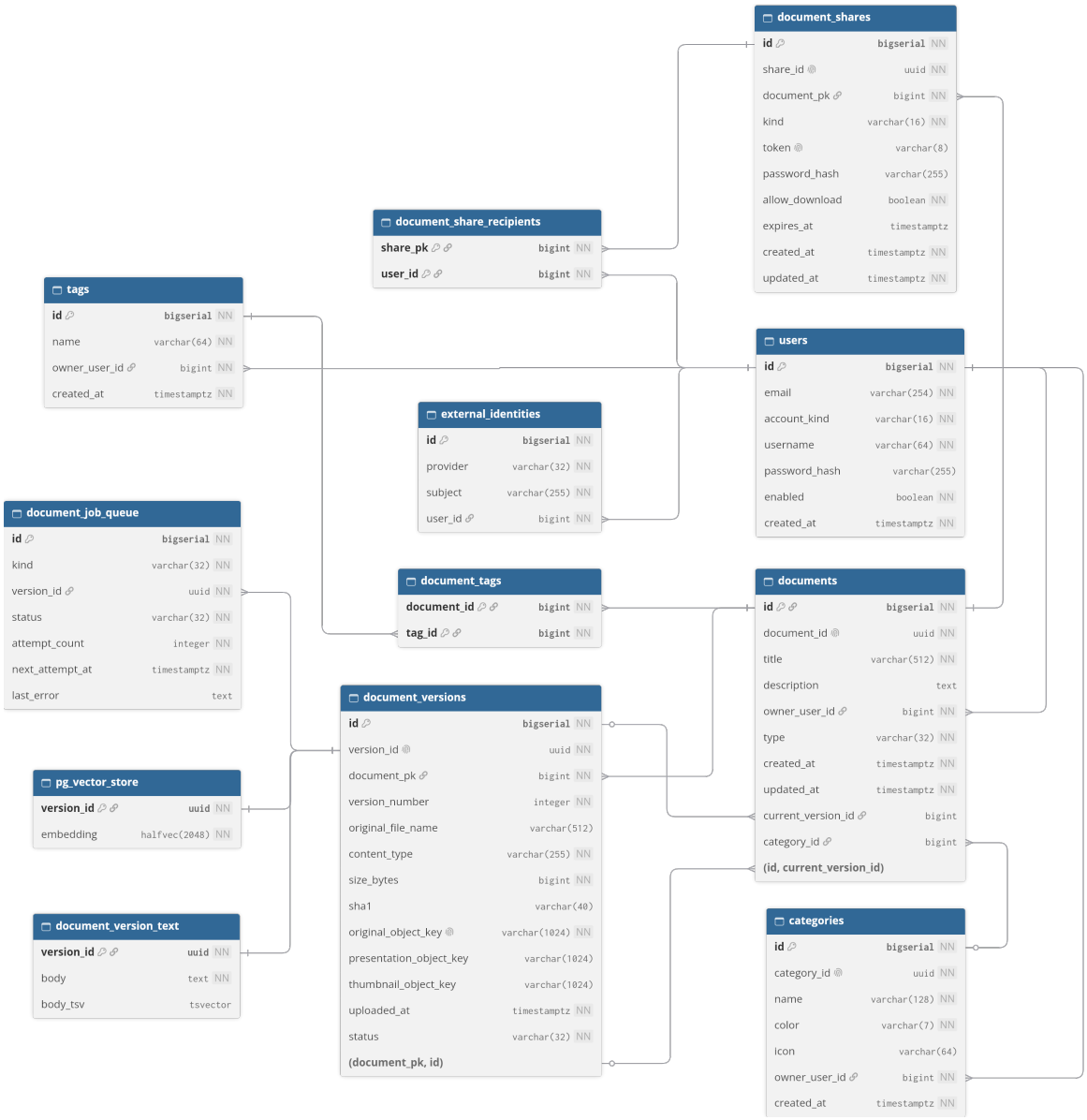
Po zapisaniu oryginału sprawdzany jest typ dokumentu. Plik w formacie biurowym przechodzi przez stan *Przekonwertowany do PDF*, a następnie osiąga stan *Gotowy do przetwarzania*. Pliki pozostałych typów przechodzą do tego stanu bez etapu konwersji.

Następnie uruchamiane są równolegle indeksowanie dokumentu i generowanie miniatury. Osią-

gnięcie stanów *Zaindeksowany* oraz *Miniatura wygenerowana* oznacza zakończenie odpowiednich operacji. Punkt synchronizacji wymaga ukończenia obu gałęzi, zanim dokument przejdzie do stanu *Przetworzony*. Diagram kończy się po osiągnięciu tego stanu i nie obejmuje późniejszych operacji na dokumencie.

5.10 Diagram bazy danych

Diagram związków encji (ang. *Entity-Relationship Diagram, ERD*) przedstawia strukturę bazy danych systemu DMS. Pokazuje tabele, ich kolumny i typy danych oraz powiązania wyrażone za pomocą kluczy głównych i obcych.



Rys. 5.16. Diagram ERD bazy danych systemu DMS

Na *rysunku 5.16* przedstawiono model bazy danych obejmujący konta użytkowników, dokumenty

i ich wersje, klasyfikację, udostępnienia oraz zadania przetwarzania. Podstawę modelu stanowią tabele *users*, *documents* i *document_versions*.

Tabela *users* przechowuje dane konta. Powiązana z nią tabela *external_identities* przechowuje identyfikatory użytkownika u zewnętrznych dostawców tożsamości. Jeden użytkownik może być właścicielem wielu dokumentów, kategorii i tagów.

Tabela *documents* zawiera metadane wspólne dla dokumentu. Kolejne wersje dokumentu są przechowywane w tabeli *document_versions*. Kolumna *current_version_id* w tabeli *documents* wskazuje bieżącą wersję. Klucze obiektów *original_object_key*, *presentation_object_key* i *thumbnail_object_key* identyfikują odpowiednio oryginał, wersję prezentacyjną oraz miniaturę w magazynie S3.

Klasyfikację dokumentów opisują tabele *categories* i *tags*. Powiązanie dokumentów z tagami ma charakter wiele-do-wielu i jest realizowane przez tabelę pośredniczącą *document_tags*.

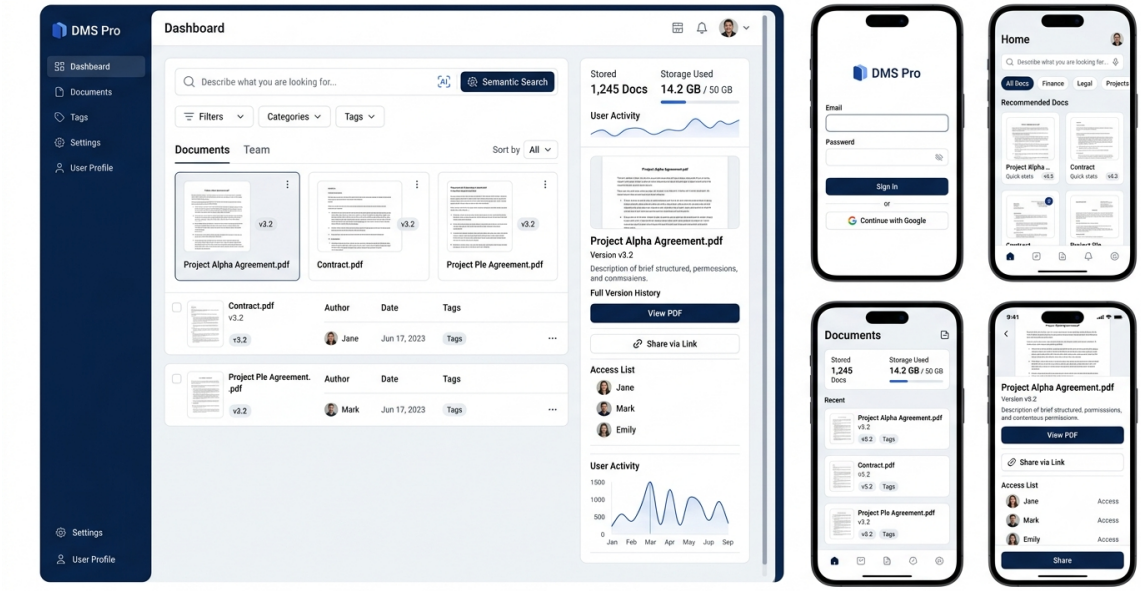
Tabela *document_shares* przechowuje dane udostępnienia. Tabela *document_share_recipients* łączy udostępnienia z kontami ich odbiorców, umożliwiając przypisanie wielu użytkowników do jednego udostępnienia.

Dane powstające podczas przetwarzania są powiązane z konkretną wersją dokumentu. Tabela *document_version_text* przechowuje wyodrębniony tekst oraz jego reprezentację *tsvector* na potrzeby wyszukiwania pełnotekstowego. Tabela *pg_vector_store* zawiera wektor osadzenia o 2048 składowych, zapisany w typie *halfvec(2048)*. W obu tabelach kluczem powiązania jest identyfikator *version_id*.

Tabela *document_job_queue* służy do obsługi zadań przetwarzania wersji dokumentów.

Dane relacyjne, tekst dokumentów i wektory osadzeń są przechowywane w jednej bazie PostgreSQL z rozszerzeniem *pgvector*. Zawartość plików oraz ich wersje prezentacyjne i miniatury znajdują się w magazynie S3, a baza danych przechowuje klucze umożliwiające ich odnalezienie.

5.11 Projekt interfejsu graficznego



Rys. 5.17. Projekt interfejsu graficznego

Projekt interfejsu graficznego systemu został opracowany z uwzględnieniem zasad nowoczesnego projektowania aplikacji internetowych i mobilnych. Głównym celem było stworzenie przejrzystego, intuicyjnego oraz estetycznego środowiska umożliwiającego efektywne zarządzanie dokumentami oraz szybki dostęp do funkcji wyszukiwania semantycznego. Interfejs użytkownika został zaprojektowany jako aplikacja internetowa z responsywnym trybem mobilnym, dostosowanym do urządzeń z ekranami dotykowymi. Dzięki temu użytkownik może korzystać z systemu w spójny sposób zarówno na komputerze, jak i na urządzeniu mobilnym. W projekcie zastosowano nowoczesną kolorystykę opartą na odcieniach granatu, bieli oraz jasnej szarości. Taki dobór barw zapewnia wysoką czytelność interfejsu oraz profesjonalny wygląd aplikacji. Dodatkowo wykorzystano zaokrąglone krawędzie komponentów, subtelne efekty cieniowania oraz przejrzystą typografię, zgodną z aktualnymi trendami projektowania systemów klasy SaaS. Główny widok aplikacji internetowej składa się z panelu nawigacyjnego, obszaru wyszukiwania oraz listy dokumentów. Użytkownik ma możliwość filtrowania zasobów według kategorii i tagów, przeglądania szczegółowych informacji o dokumentach, zarządzania wersjami plików oraz udostępniania dokumentów za pomocą generowanych linków. W interfejsie przewidziano również podgląd dokumentów oraz dostęp do funkcji pobierania i konwersji plików. W mobilnym widoku aplikacji internetowej kluczowe funkcjonalności systemu zostały umieszczone w łatwo dostępnych miejscach ekranu, a nawigację zrealizowano za pomocą dolnego paska menu. Użytkownik może wyszukiwać dokumenty, przeglądać ich zawartość, zarządzać metadanymi oraz udostępniać zasoby bez konieczności korzystania z komputera. *Na rysunku 5.17* przedstawiono wizualizację interfejsu aplikacji internetowej w widoku komputerowym oraz mobilnym. Projekt stanowi koncepcję wyglądu końcowego systemu i prezentuje sposób rozmieszczenia najważniejszych funkcji dostępnych dla użytkownika.

Bibliografia

[1] S. Koyuncu Tunç, „Comparative Analysis of Four Usability Assessment Techniques for Electronic Record Management Systems,” *Advances in Human-Computer Interaction*, t. 2025, nr 1, 2025, **artno** 8693889. DOI: 10.1155/ahci/8693889 URL: https://onlinelibrary.wiley.com/doi/10.1155/ahci/8693889

[2] Paperless-ngx contributors. „Paperless-ngx: Basic Usage,” dostęp 29 października 2026. URL: https://docs.paperless-ngx.com/usage/

[3] OpenKM. „OpenKM: Document Management,” dostęp 29 października 2026. URL: https://www.openkm.com/en/document-management.html

[4] M-Files. „M-Files: AI-Powered Document Management System,” dostęp 29 października 2026. URL: https://www.m-files.com/

[5] Qwen Team. „Qwen3-VL-Embedding: Evaluation Results on MMEB-V2,” dostęp 29 października 2026. URL: https://github.com/QwenLM/Qwen3-VL-Embedding#evaluation-results-on-mmeb-v2

[6] S. Aigner, R. Elizarov, S. Isakova i D. Jemerov, *Kotlin in Action*, 2 wyd. Manning Publications, 2024, ISBN: 9781617299605. URL: https://www.manning.com/books/kotlin-in-action-second-edition

[7] C. Walls, *Spring in Action*, 6 wyd. Manning Publications, 2022, ISBN: 9781617297571. URL: https://www.manning.com/books/spring-in-action-sixth-edition

[8] S. P. Kane i K. Matthias, *Docker: Up & Running: Shipping Reliable Containers in Production*, 3 wyd. O'Reilly Media, 2023, ISBN: 9781098131814. URL: https://www.oreilly.com/library/view/docker-up/9781098131814/

[9] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*, 1 wyd. O'Reilly Media, 2017, ISBN: 9781449373320. URL: https://www.oreilly.com/library/view/designing-data-intensive-applications/9781491903063/

[10] C. D. Manning, P. Raghavan i H. Schütze, *Introduction to Information Retrieval*, 1 wyd. Cambridge University Press, 2008, ISBN: 9780521865715. URL: https://nlp.stanford.edu/IR-book/information-retrieval-book.html

[11] Y. A. Malkov i D. A. Yashunin, „Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, t. 42, nr 4, s. 824–836, 2020. DOI: 10.1109/TPAMI.2018.2889473 URL: https://arxiv.org/abs/1603.09320

[12] G. V. Cormack, C. L. A. Clarke i S. Büttcher, „Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods," w: *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, Association for Computing Machinery, 2009, s. 758–759. DOI: 10.1145/1571941.1572114 URL: https://cormack.uwaterloo.ca/cormack/cormacksigir09-rrf.pdf

[13] Z. Jiang, R. Meng, X. Yang, S. Yavuz, Y. Zhou i W. Chen, „VLM2Vec: Training Vision-Language Models for Massive Multimodal Embedding Tasks," w: *The Thirteenth International Conference on Learning Representations*, 2025. URL: https://proceedings.iclr.cc/paper_files/paper/2025/file/04261fce1705c4f02f062866717d592a-Paper-Conference.pdf

[14] R. T. Fielding, „Architectural Styles and the Design of Network-based Software Architectures," rozprawa doktorska, University of California, Irvine, 2000. URL: https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

[15] Spring. „Spring Boot: Reference Documentation," dostęp 29 października 2026. URL: https://docs.spring.io/spring-boot/reference/

[16] Spring. „Spring Framework: Spring Web MVC," dostęp 29 października 2026. URL: https://docs.spring.io/spring-framework/reference/web/webmvc.html

[17] Spring. „Spring Security: Cross Site Request Forgery (CSRF)," dostęp 29 października 2026. URL: https://docs.spring.io/spring-security/reference/servlet/exploits/csrf.html

[18] Docker Inc. „Dockerfile Reference," dostęp 29 października 2026. URL: https://docs.docker.com/reference/dockerfile/

[19] Docker Inc. „Docker Compose Documentation," dostęp 29 października 2026. URL: https://docs.docker.com/compose/

[20] PostgreSQL Global Development Group. „PostgreSQL 18 Documentation: Full Text Search," dostęp 29 października 2026. URL: https://www.postgresql.org/docs/18/textsearch.html

[21] pgvector contributors. „pgvector: Open-source Vector Similarity Search for Postgres," dostęp 29 października 2026. URL: https://github.com/pgvector/pgvector

[22] Amazon Web Services. „Amazon S3: Download and Upload Objects with Presigned URLs," dostęp 29 października 2026. URL: https://docs.aws.amazon.com/AmazonS3/latest/userguide/using-presigned-url.html

[23] MinIO Inc. „MinIO: S3-compatible Object Storage. Repository and Documentation," dostęp 29 października 2026. URL: https://github.com/minio/minio

[24] F5 Inc. „nginx: Module ngx_http_proxy_module,” dostęp 29 października 2026. URL: https://nginx.org/en/docs/http/ngx_http_proxy_module.html

[25] React contributors. „React: Quick Start,” dostęp 29 października 2026. URL: https://react.dev/learn

[26] Vercel Inc. „Next.js: App Router Documentation,” dostęp 29 października 2026. URL: https://nextjs.org/docs/app

[27] Orval contributors. „Orval: React Query,” dostęp 29 października 2026. URL: https://orval.dev/docs/guides/react-query/

[28] OpenAPI Initiative. „OpenAPI Specification v3.1.0,” dostęp 29 października 2026. URL: https://spec.openapis.org/oas/v3.1.0.html